GIFT GALLERY: Gift and Accessories Store

SOFTWARE DESIGN DOCUMENT

LAIBA YASEEN
ROLLNO:22323555083 (BSCS) (SESSION: 2023-2025)
Khawaja Fareed Govt Post Graduate College, Rahim Yar Khan

Revision History

Date	Description	Author	Comments
<*date*>		Laiba Yaseen	

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	SIR USMAN GAHNNI	Supervisor, CSIT 21306	<date>

Table of Contents

1 INTRODUCTION

1.1 PURPOSE

The purpose of this Software Design Document (SDD) is to provide a comprehensive overview of the design and architecture of an GiftGallery (Gift and Accessories Store) System. It describes the structure, components, and interactions of the system to facilitate the development process.

1.2 Scope

The GiftGallery (Gift and Accessories Store) System allows users to browse, search, and purchase gifts online. It includes functionalities such as user registration, gift catalog management, shopping cart, payment processing, order management, and user reviews.

1.3 Definitions, Acronyms and Abbreviations:

- **GAAS**: Gift And Accessories Store
- **SDD**: Software Design Document
- **UI**: User Interface
- **UX**: User Experience
- **API**: Application Programming Interface
- **RDBMS**: Relational Database Management System
- **HTML**: Hypertext Markup Language
- **CSS**: Cascading Style Sheets
- **JS**: JavaScript
- **HTTP**: Hypertext Transfer Protocol
- **AWS**: Amazon Web Services
- **GCP**: Google Cloud Platform
- **SQL**: Structured Query Language
- **Git**: Version control system for tracking changes in source code
- **GitHub**: Web-based Git repository hosting service
- **PCI DSS**: Payment Card Industry Data Security Standard
- **CMS**: Content Management System
- **CRM**: Customer Relationship Management
- **SEO**: Search Engine Optimization

- **GDPR**: General Data Protection Regulation

1.4 References:

- IEEE Standard 1016-2009, IEEE Standard for Information Technology – Systems Design – Software Design Description, IEEE Computer Society, 2009.
- "Design Patterns: Elements of Reusable Object-Oriented Software" by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides.
- "Clean Architecture: A Craftsman's Guide to Software Structure and Design" by Robert C. Martin.
- "Domain-Driven Design: Tackling Complexity in the Heart of Software" by Eric Evans.
- "Software Architecture in Practice" by Len Bass, Paul Clements, Rick Kazman.
- "Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions" by Gregor Hohpe and Bobby Woolf.

1.5 Overview:

This Software Design Document (SDD) presents the design and architecture of a GiftGallery (Gift and Accessories Store) system. It includes components such as user interface, authentication, catalog management, shopping cart, payment processing, order management, and user reviews. The system follows a three-tier architecture, with a presentation tier, application tier, and data tier. The SDD provides class diagrams and sequence diagrams to illustrate the relationships and interactions between system components.

2 System Overview:

The GiftGallery (Gift and Accessories Store) system consists of the following components:

- User Interface: Provides a web-based interface for users to interact with the system.
- Database: Stores information about gifts and accessories, users, orders, and other relevant data.
- Authentication: Handles user registration, login, and authentication processes.
- Catalog Management: Manages the gifts and accessories catalog, including adding, editing, and deleting gifts.
- Shopping Cart: Allows users to add and remove gifts and accessories from their shopping cart.
- Payment Processing: Handles payment transactions securely.
- Order Management: Tracks and manages user orders.
- User Reviews: Enables users to provide feedback and reviews for gifts and accessories.

2.1 General Constraints:

A GiftGallery (Gift and Accessories Store) is subject to various constraints that must be considered during its design and development. Security is a crucial constraint, as the system must prioritize protecting user information, such as login credentials and payment details, through secure authentication mechanisms, data encryption, and safeguards against unauthorized access. Performance is another important constraint, requiring the GAAS to deliver a responsive and efficient user experience, minimizing page load times, and effectively managing high volumes of concurrent users and transactions.

Scalability is essential to accommodate future growth, enabling the system to handle increasing user demands, a growing catalog of gifts and accessories, and potential spikes in traffic without compromising performance. Compatibility is a constraint that necessitates ensuring the GAAS works seamlessly across different web browsers, devices, and operating systems, maximizing accessibility for a

diverse user base. Lastly, regulatory compliance is a critical consideration, ensuring the GAAS adheres to relevant laws and regulations, such as data protection and privacy guidelines, to safeguard user data and maintain legal compliance.

2.2 Application Overview:

The GiftGallery (Gift and Accessories Store) application is designed to provide users with a convenient and user-friendly platform to browse search, and purchase books online. The application offers a range of features and functionalities to enhance the book shopping experience.

User Registration and Authentication:

Users can create an account by registering with their personal information. The application ensures secure authentication processes to protect user credentials and prevent unauthorized access.

Gifts Catalog Management:

The GAAS application manages a comprehensive catalog of gifts and accessories, including their titles, brand, price, descriptions, and cover images. The catalog is regularly updated to provide an up-to-date selection of gift and accessories for users to explore.

Search and Filtering:

The application incorporates a powerful search functionality that enables users to search for gifts and accessories based on various criteria, such as title, author, genre, or keywords. Additionally, users can apply filters to refine their search results based on preferences like price range, publication date, or customer ratings.

Details and Reviews:

Each gift in the catalog is accompanied by detailed information, including a synopsis, author bio, and customer reviews. Users can read reviews and ratings submitted by other customers to make informed decisions about their gift purchases.

Shopping Cart and Checkout:

The GAAS application allows users to add books to their shopping cart while browsing the catalog. Users can review and modify the contents of their cart, update quantities, and apply promotional discounts if available. During the checkout process, users provide shipping details and select a preferred payment method.

Payment Processing:

The application integrates with secure payment gateways to facilitate smooth and secure online transactions. Users can make payments using various methods, such as credit/debit cards, digital wallets, or other available payment options.

Order Management and Tracking:

After a successful purchase, users receive an order confirmation along with a unique order ID. The GAAS application tracks and manages orders, allowing users to view their order history and track the status of their shipments.

User Reviews and Recommendations:

Users can provide feedback and reviews for gifts they have purchased. The application may also provide personalized book recommendations based on user's browsing history, purchase patterns, and preferences.

The GiftGallery (gifts and accessories Store) application aims to deliver an intuitive and seamless experience for users, facilitating easy book discovery, convenient purchasing, and reliable order management.

2.3 Design Language:

The design language of the have GiftGallery (gifts and accessories Store) prioritized:

- User-friendly interface with clear navigation and intuitive functionality.
- Consistent branding elements, including logo, color scheme, and typography.
- Responsive design for optimal viewing across different devices.
- Visual appeal through high-quality images, appealing typography, and engaging layouts.

- Gift-centric elements, intuitive search and filtering, visual feedback, and accessibility considerations to enhance the overall user experience.

2.4 Assumptions:

Assumptions for the GiftGallery (gifts and accessories Store) SDD:

- Users have access to a compatible web browser or mobile device with internet connectivity.
- GAAS utilizes a relational database management system for data storage.
- Integration with third-party payment gateways for secure online transactions.
- Secure authentication mechanisms and data encryption for user account protection.
- Up-to-date gift catalog management with regular updates.
- Utilization of third-party APIs for additional book information or features.
- Design considerations for scalability and performance.
- These assumptions serve as a foundation for designing the GAAS application and can be further refined as needed during the development process.

2.5 System Environment:

System Environment of GiftGallery (gifts and accessories Store)

Operating System:

The GAAS can be designed to run on multiple operating systems, such as Windows, macOS, and Linux, ensuring compatibility across a wide range of user devices.

Web Server:

The GAAS can be deployed on a web server, such as Apache Tomcat, Nginx, or Microsoft IIS, to handle HTTP requests and serve the application to users.

Programming Languages and Frameworks:

The GAAS can be developed using programming languages like JavaScript, Python, or Java. Frameworks such as Node.js, Django, or Spring can be employed to streamline development and provide additional functionality.

Database Management System:

The GAAS can utilize a relational database management system (RDBMS) like MySQL, PostgreSQL, or Oracle to store and manage data related to books, users, orders, and other relevant information.

Web Technologies:

The GAAS can leverage web technologies such as HTML, CSS, and JavaScript to create interactive and responsive user interfaces. AJAX and RESTful APIs can facilitate asynchronous communication between the client and server.

Security Measures:

The GAAS should incorporate security measures such as SSL/TLS encryption to ensure secure communication between users and the application. Proper user authentication and authorization mechanisms should be implemented to protect sensitive data.

Cloud Services (Optional):

The GAAS can utilize cloud services like Amazon Web Services (AWS), Microsoft Azure, or Google Cloud Platform (GCP) for hosting, scalability, and data storage. Cloud-based infrastructure can provide flexibility and reliability for the OBS.

Browser Compatibility:

The GAAS should be designed and tested to ensure compatibility with popular web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. Cross-browser testing can help ensure a consistent experience for users.

The GAAS system environment encompasses the necessary components and technologies to support the development, deployment, and operation of the Online Book Store application, ensuring its functionality, security, and compatibility across various platforms and devices.

3 System Architecture:

3.1 Architectural Design:

The architectural design of the GiftGallery (gifts and accessories Store) follows a three-tier layered architecture: presentation layer (front-end), business logic layer, and data storage layer (database). The presentation layer handles the user interface, the business logic layer manages the application's functionality and integration with external services, and the data storage layer manages data storage and retrieval. This design promotes modularity, scalability, and easy maintenance.

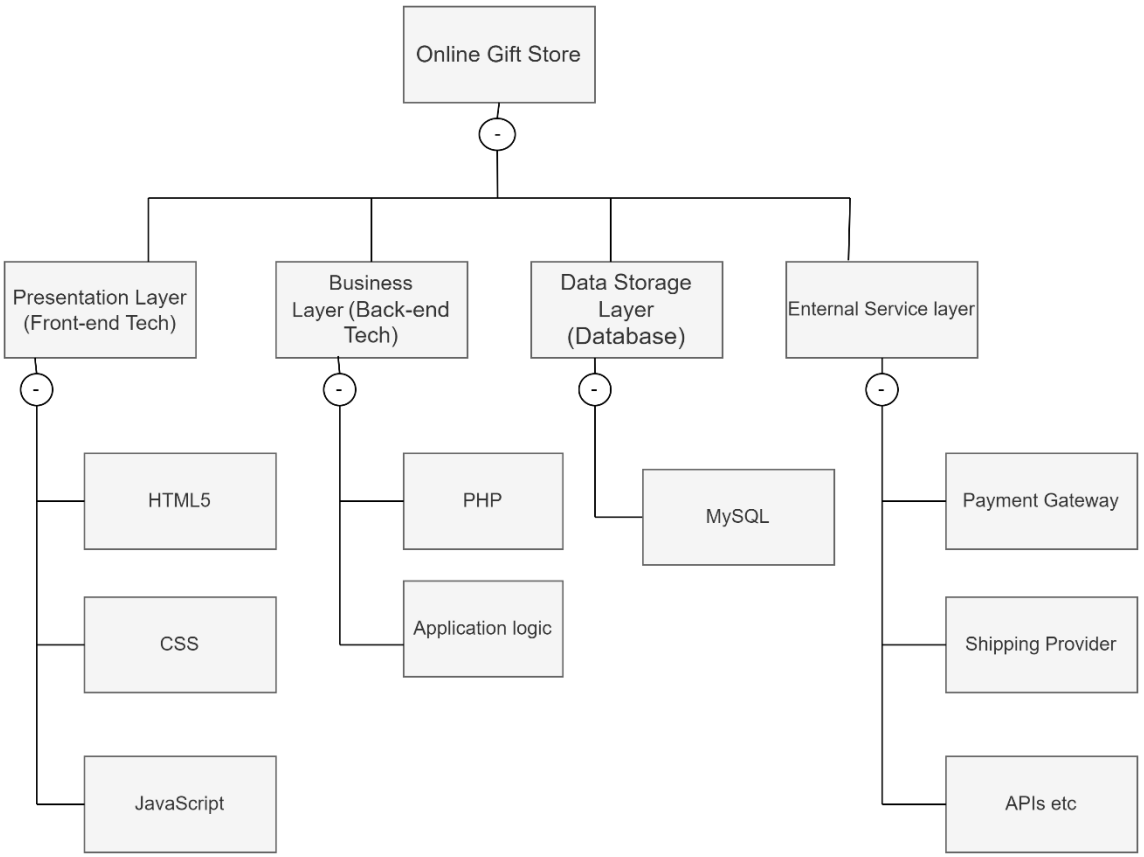


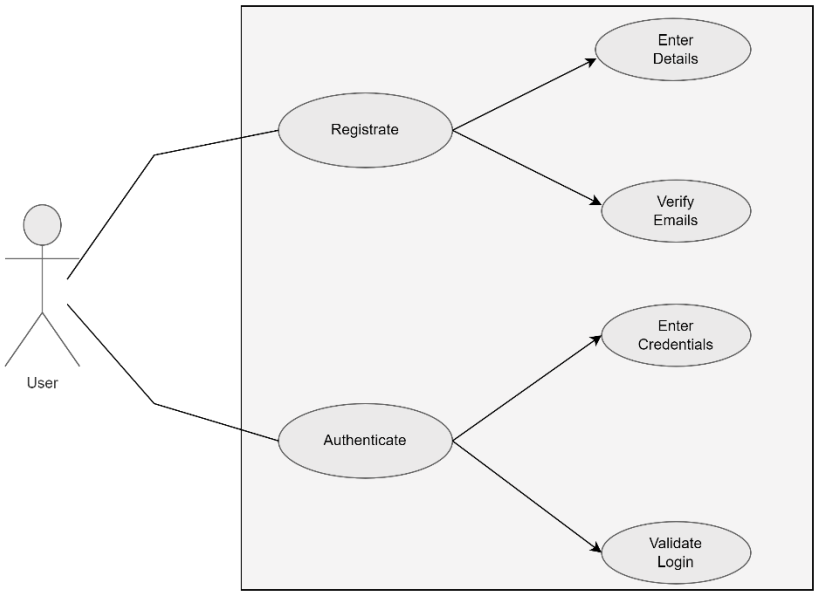
Figure 1- Block Diagram

3.2 Decomposition Description:

- **Presentation Layer Decomposition:** The Presentation Layer can be further decomposed into sub-components responsible for different aspects of the user interface. This may include components for product listing, product details, shopping cart, user authentication, and user profile management. Each sub-component handles specific functionalities and interacts with the Business Logic Layer to retrieve or update data.
- **Business Logic Layer Decomposition:** The Business Logic Layer can be decomposed into modules or services that handle different business processes of the GAAS. For example, there may be modules for product management, order processing, inventory management, customer management, and payment processing. Each module encapsulates the logic and functionality related to its specific area and communicates with the Presentation Layer for user interactions and the Data Storage Layer for data retrieval or updates.
- **Data Storage Layer Decomposition**: The Data Storage Layer can be decomposed into multiple database tables or entities, each representing a specific data entity in the GAAS, such as products, customers, orders, and reviews. This decomposition allows for efficient storage, retrieval, and manipulation of data. Additionally, the database layer may include data access components or services that handle database connectivity and query execution.
- **External Services Decomposition**: The External Services block can be decomposed into separate components or integration modules for different third-party services. This may include modules for payment gateway integration, shipping provider integration, and other external APIs used for functionalities such as geolocation, analytics, or marketing. Each integration module is responsible for communicating with the respective external service and handling the necessary data exchange.

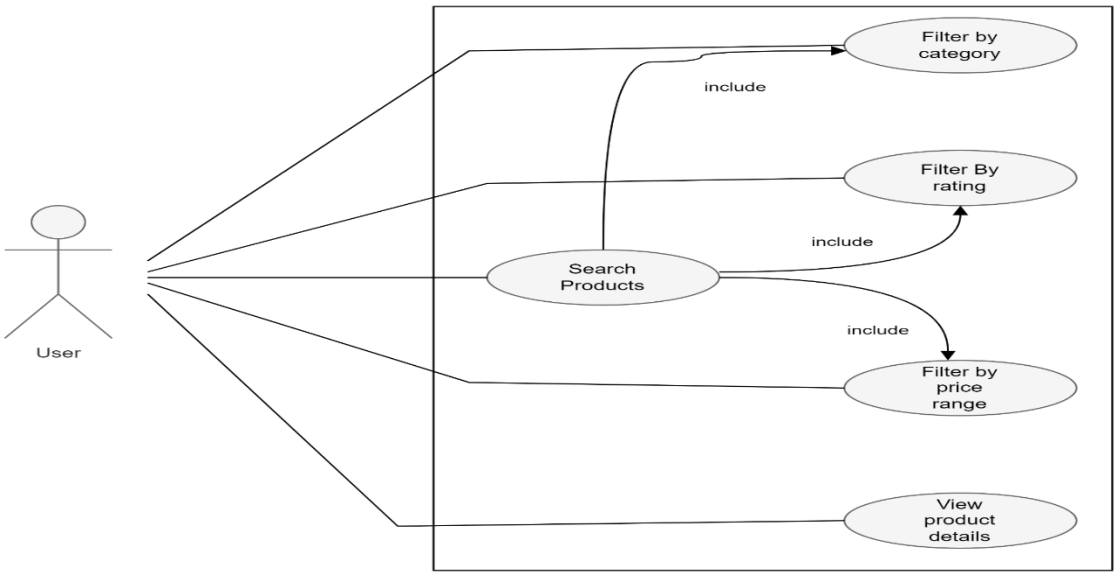
3.2.1 Use Cases

3.2.1.1 USER REGISTRATION



Use Case 1- Registration

3.2.1.2 USER PRODUCT SEARCH AND FILTERING



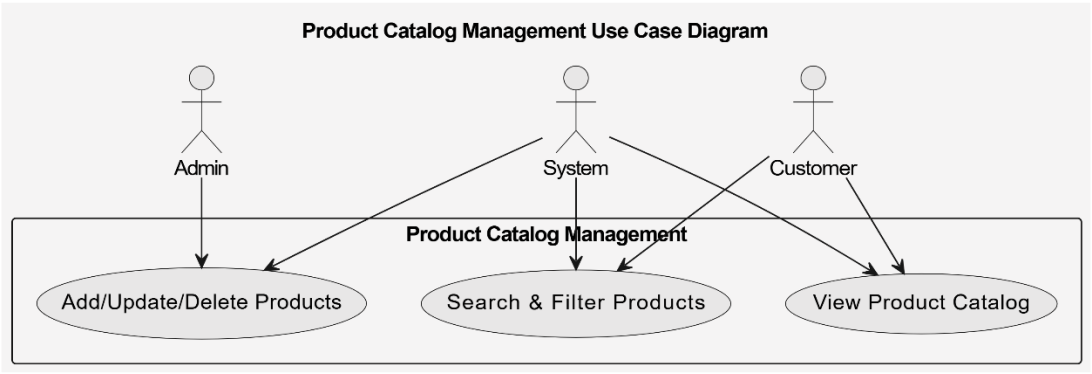
Use Case 2- User Product Search and Filtering

Actor	*User*
Pre-condition	*User is on the product catalog or search page.* *Search results or product list is displayed.*
Post-condition	*Relevant product results are displayed.*

	Filtered product list is shown to the user.
Steps	*1. User enters search keywords.* *2. System retrieves matching products.* *3. Results are displayed.*

ACTOR	User
PRE-CONDITION	The user has registered an account on the website.
POST-CONDITION	The user has successfully logged into their account.
STEPS	1. Navigate to the website's login page. 2. Enter the email address and password associated with the user's account. 3. Click the "Login" button. 4. If the email address and password are correct, the user will be redirected to their account page. 5. If the email address and password are incorrect, the user will receive an error message and will need to try again.

3.2.1.3 PRODUCT CATALOG



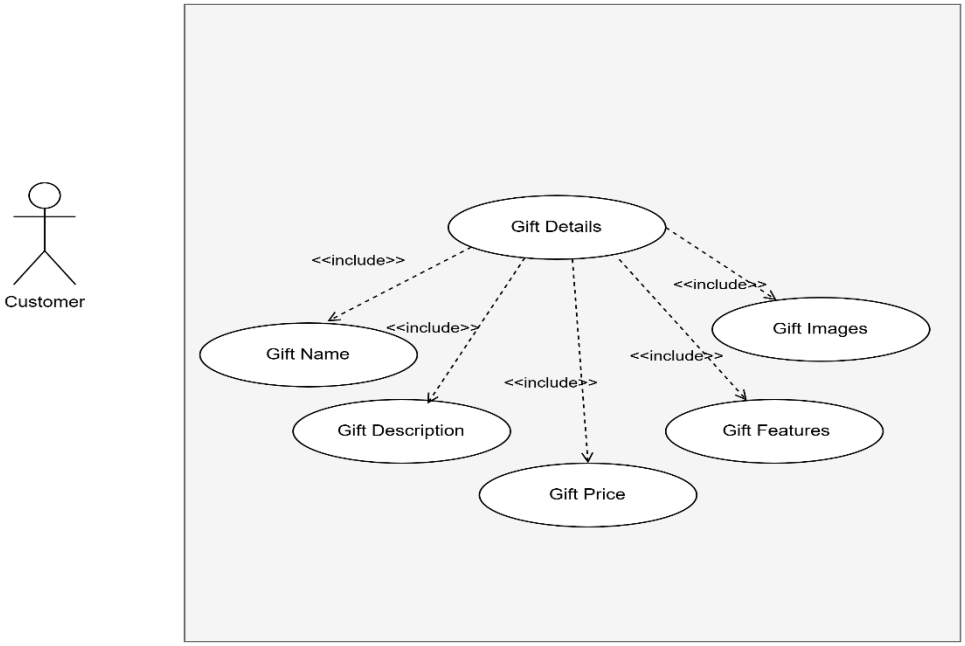
Use Case 3- Product Catalog

ACTOR	Customer

PRE-CONDITION	The customer has successfully logged into their account.
POST-CONDITION	The customer has viewed the available products in the catalog.
STEPS	1. Navigate to the website's product catalog page. 2. Browse or search for the desired products. 3. Click on a product to view its details, such as description, price, and availability.

ACTOR	Admin
PRE-CONDITION	The admin has successfully logged into the admin panel.
POST-CONDITION	The admin has added, edited, or removed products from the product catalog.
STEPS	1. Navigate to the product catalog management section in the admin panel. 2. Click "Add Product" to add a new product, or click "Edit" next to an existing product to modify its details. 3. Fill in the product details, such as title, author, description, price, and availability. 4. Upload any relevant images or files. 5. Save the changes to the product. 6. If necessary, click "Delete" next to a product to remove it from the catalog. 7. Repeat steps 2-6 for any additional products to be managed. 8. Logout from the admin panel when done.

3.2.1.4 PRODUCT DETAILS

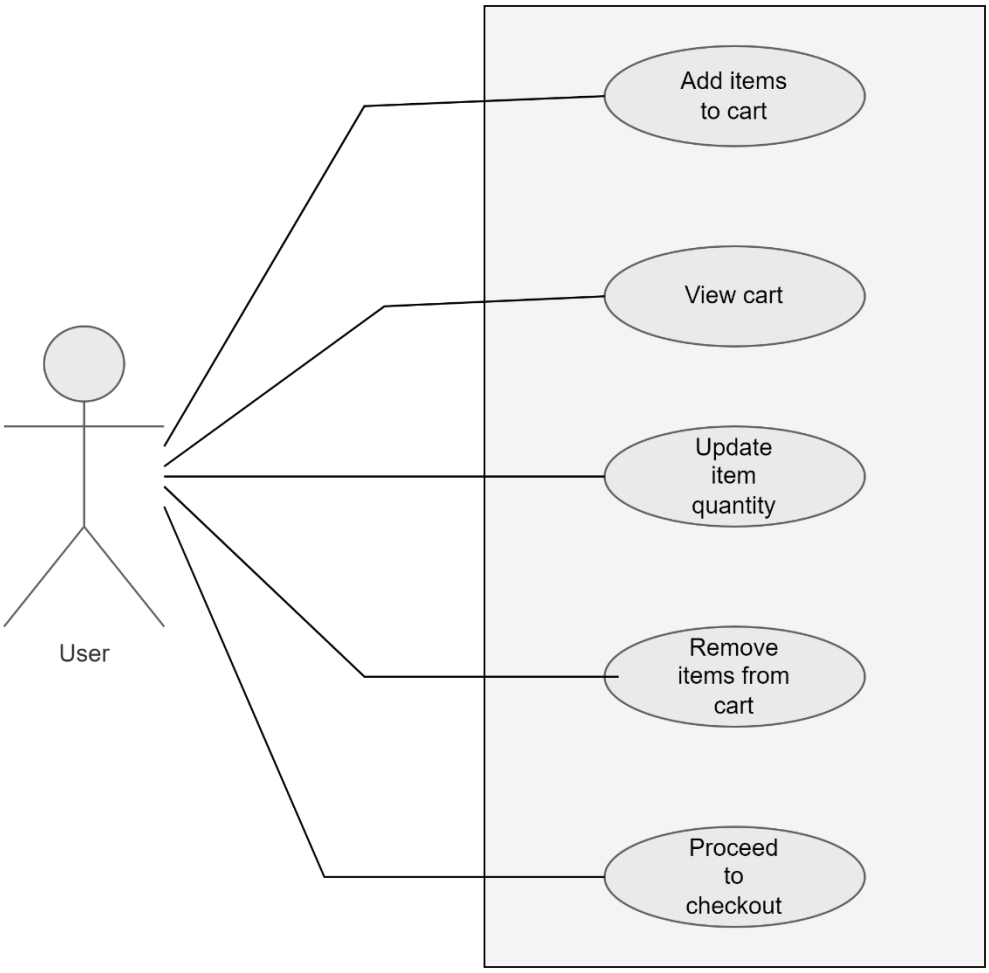


Use Case 4- Product Details

ACTOR	Customer
PRE-CONDITION	The customer has accessed the product catalog or search results page.
POST-CONDITION	The customer has viewed the details of the selected product.
STEPS	1. Browse or search for the desired product in the product catalog or search results page. 2. Click on the product's title or image to view its details page. 3. Read the product description, author, publication details, and any other relevant information. 4. View the product's price and availability. 5. Select any desired options, such as format or quantity. 6. Add the product to the shopping cart if interested in purchasing. 7. If not interested in purchasing, use the

	browser's back button to return to the previous page.

3.2.1.5 SHOPPING CART

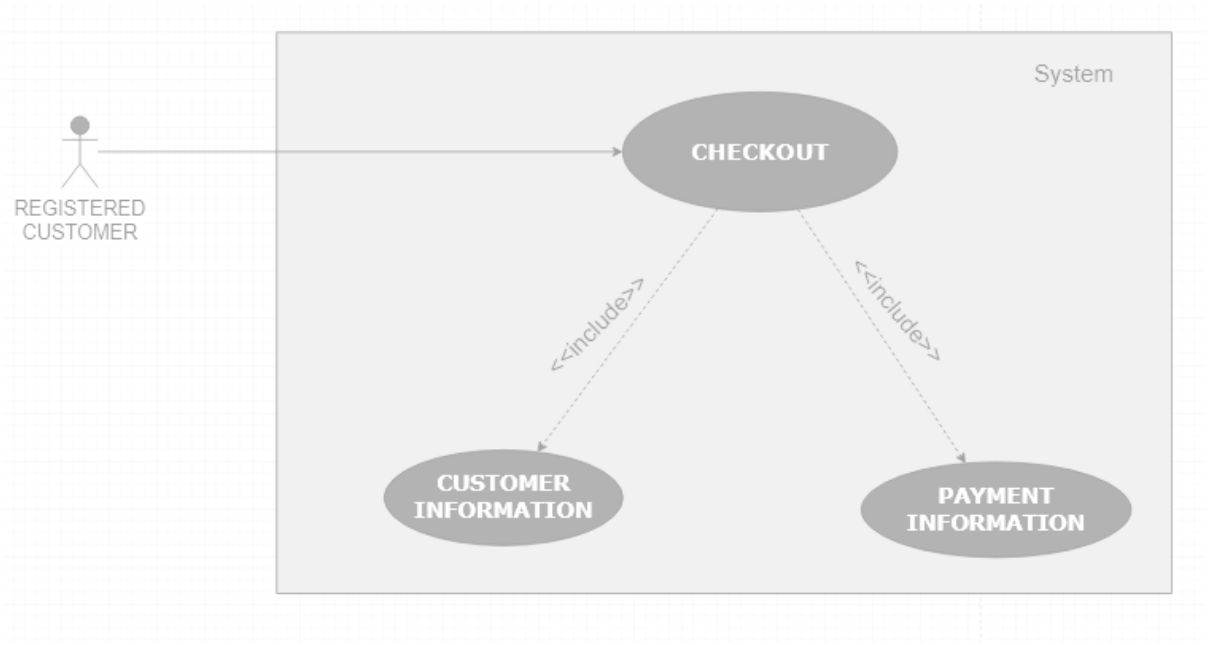


Use Case 5 - Shopping Cart

ACTOR	Customer
PRE-CONDITION	The customer has added one or more products to their shopping cart.
POST-CONDITION	The customer has reviewed and modified the contents of their shopping cart.
STEPS	1. Click on the shopping cart icon or link on the website. 2. View the list of products currently in the shopping cart.

	3. Modify the quantity or remove any products as desired. 4. Review the subtotal and any applicable taxes or fees. 5. Proceed to checkout if ready to purchase. 6. If not ready to purchase, continue browsing or return to the previous page.

3.2.1.6 CHECKOUT PROCESS:

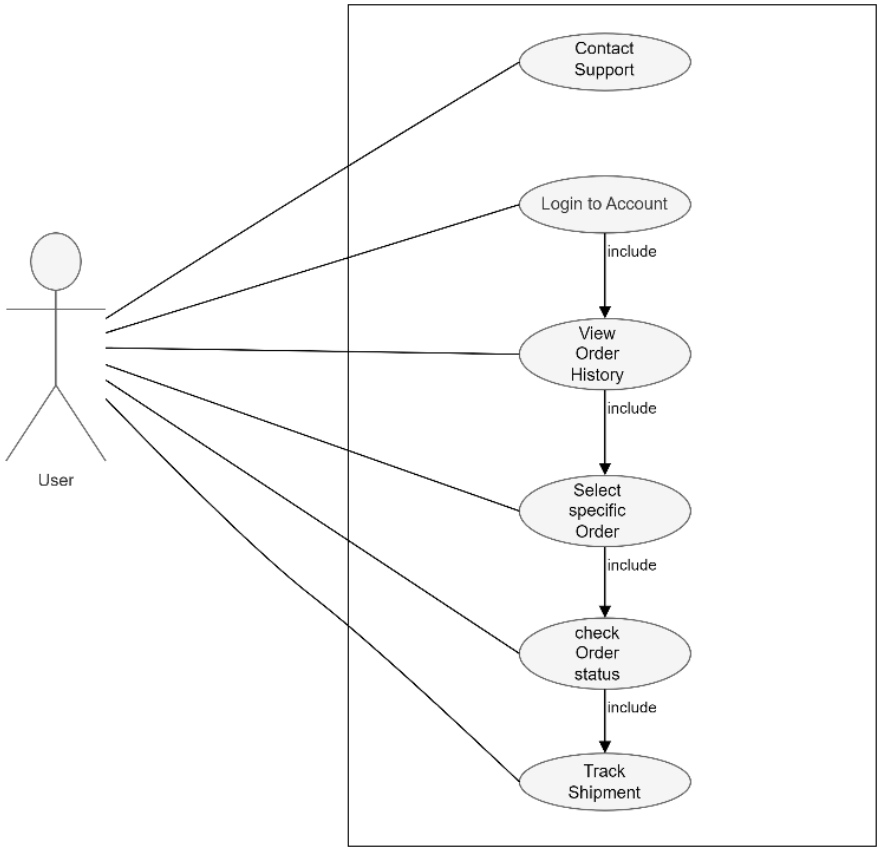


Use Case 6 - Checkout Process

ACTOR	User
PRE-CONDITION	The user has added one or more gifts to their shopping cart and is ready to proceed to checkout.
POST-CONDITION	The user has successfully completed the checkout process and the order has been placed.
STEPS	1. The user clicks on the "Checkout" button in their shopping cart. 2. The online giftstore displays a summary of the user's order, including the gifts in their

	cart and the total price. 3. The user can review the order details and make changes if necessary.

3.2.1.7 ORDER MANAGEMENT



Use Case 7 - Order Management

ACTOR	Admin
PRE-CONDITION	The admin is logged in to the online Gift And Accessories store website and has access to the order

	management system.
POST-CONDITION	he admin has successfully managed the order and updated its status.
STEPS	1. The admin logs in to the online Gift And Accessories Store website and navigates to the order management system. 2. The Online Gift And Accessories Store displays a list of all orders, sorted by date or order number. 3. The admin selects the order they want to manage from the list. 4. The Online Gift And Accessories store displays the order details, including the books in the order, the customer's shipping and billing information, and the order status. 5. The admin can update the order status by selecting from a dropdown menu of available status options (e.g. processing, shipped, delivered, cancelled, etc.). 6. If the admin updates the order status to "shipped," they can enter a tracking number or carrier information 7. The Online Gift And Accessories Store updates the order status in real-time and sends a notification to the customer via email or text message. 8. If the admin needs to contact the customer about the order (e.g. to confirm a shipping address, update them on the status, etc.), they can click on the customer's name or email address to open a communication channel. 9. The admin can view the order history, including any changes made to the order status or customer information. 10. If the admin needs to refund or cancel the

	order, they can select the appropriate option from the dropdown menu and follow the online giftstore's refund or cancellation process. 11. The Online Gift And Accessories Store updates the customer's account and sends a notification to the customer via email or text message.

ACTOR	User
PRE-CONDITION	The user has placed an order on the Online Gift And Accessories Store website and wants to check its status or manage it in some way.
POST-CONDITION	The user has successfully managed the order and updated its status or taken other relevant actions.
STEPS	1. The user logs in to the Online Gift And Accessories Store website and navigates to their account dashboard. 2. The Online Gift And Accessories Store displays a list of the user's orders, sorted by date or order number. 3. The user selects the order they want to manage from the list. 4. The online giftstore displays the order details, including the gifts in the order, the shipping and billing information, and the current order status. 5. The user can view the order history, including any changes made to the order status or customer information. 6. If the user wants to cancel the order, they can click on the "Cancel Order" button (if available) and follow the online giftstore's cancellation process.

	7. If the user wants to change the shipping address, they can click on the "Edit Shipping Address" button (if available) and follow the onlinegiftstore's address update process. 8. If the user wants to track the shipment, they can click on the "Track Shipment" button (if available) and view the current tracking status. 9. If the user needs to contact the online giftstore about the order (e.g. to request a refund or update the order status), they can click on the "Contact Customer Support" button (if available) and follow the online bookstore's customer support process. 10. The online giftstore updates the user's account and sends a notification to the user via email or text message.

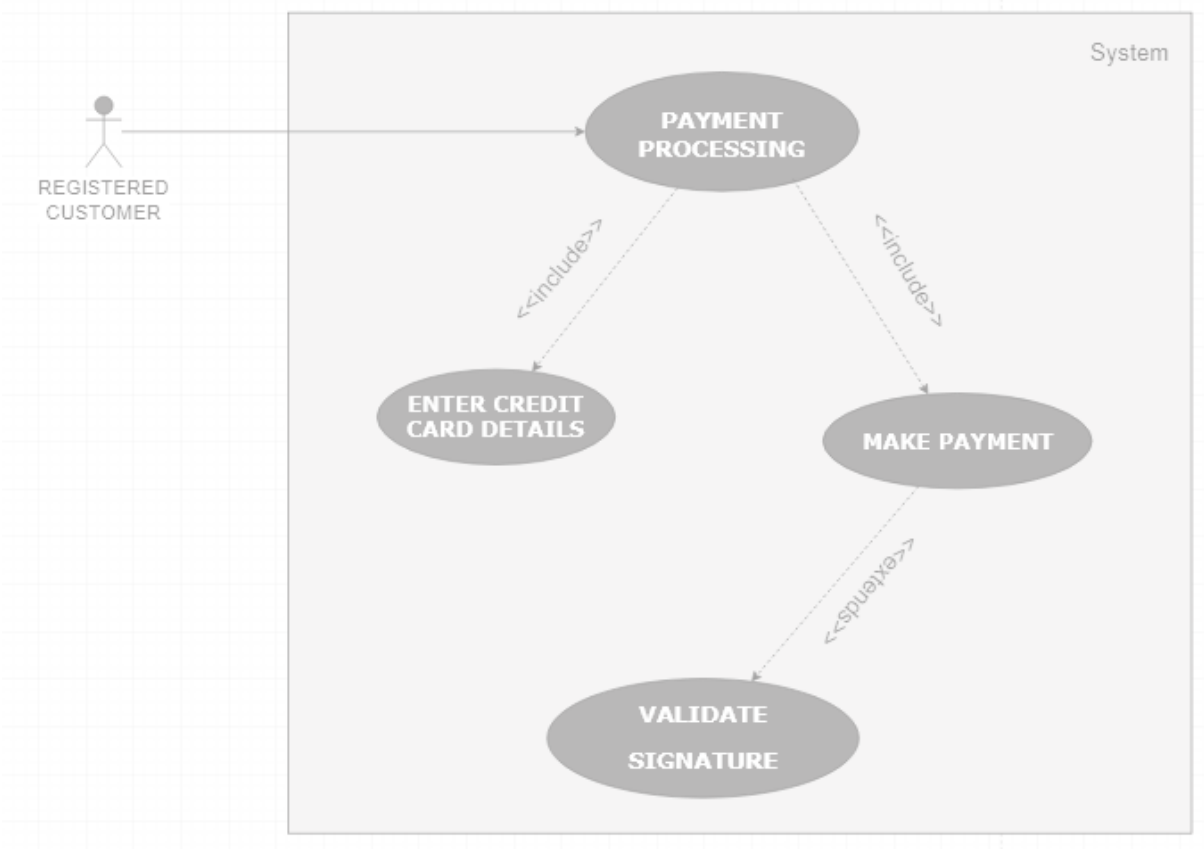
3.2.1.8 CUSTOMER SERVICES:

Use Case 8 - Customer Services

ACTOR	The actor in this case would be the admin of the Online Gift And Accessories Store. The admin is responsible for managing and overseeing the operations of the Online Gift And Accessories Store.
PRE-CONDITION	The pre-condition for providing customer service as an admin in an Online Gift And Accessories Store would be that the customer has a question, concern or issue that needs to be addressed.
POST-CONDITION	The post-condition for providing customer service as an admin in an Online Gift And Accessories Store would be that the customer's question, concern or issue has been resolved to their satisfaction.
STEPS	1. Listen to the customer's concern: When a customer reaches out with a question or issue, it's important to listen carefully to

	what they are saying. Take the time to fully understand their concern before responding. 2. Acknowledge the customer's concern: Once you've listened to the customer, acknowledge their concern and let them know that you understand their frustration or confusion. 3. Provide a solution: After you've understood the customer's concern and acknowledged it, work on providing a solution that meets their needs. This could include answering a question, providing additional information, or resolving an issue. 4. Offer alternatives: If the solution you provide isn't what the customer is looking for, offer alternatives that may be more suitable. 5. Follow up: Once the customer's concern has been addressed, follow up with them to ensure that they are satisfied with the solution provided. This helps to build trust and demonstrates your commitment to customer service. 6. Document the interaction: Finally, it's important to document the customer's concern and the steps taken to address it. This can be useful in future interactions with the customer or in tracking common issues that arise in the Online Gift And Accessories Store

3.2.1.9 Payment Processing:

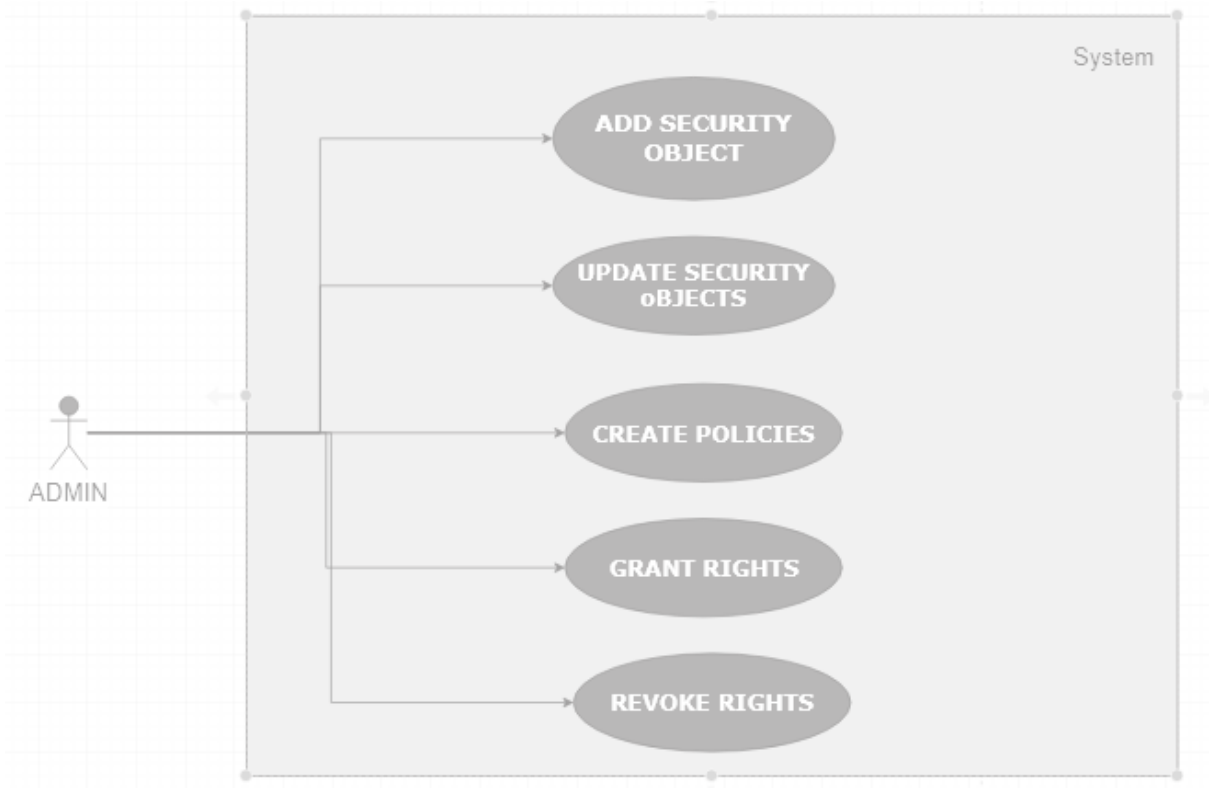


Use Case 9 - Payment Services

ACTOR	User
PRE-CONDITION	The user has selected the gift(s) they wish to purchase and has reached the payment page.
POST-CONDITION	The user has successfully completed the payment process.
STEPS	1. The user selects the preferred payment method, such as credit card, debit card, net banking, or digital wallets. 2. The user enters the payment details, such as card number, CVV, expiration date, or account details. 3. The system validates the payment details and checks for any errors or discrepancies. 4. If the payment details are valid, the system processes the payment and charges the

	corresponding amount. 5. The system generates a payment receipt or confirmation page, which displays the details of the transaction, such as the transaction ID, amount charged, date and time of transaction, and the user's name and address. 6. The system sends an email notification to the user with the payment receipt or confirmation. 7. If the payment is successful, the system updates the inventory status to reflect the gifts that have been sold. 8. The user is redirected to a page that displays the details of their purchase, such as the gift title, author name, quantity, total amount charged, and estimated delivery date. 9. The user can either continue shopping or log out of the online giftstore

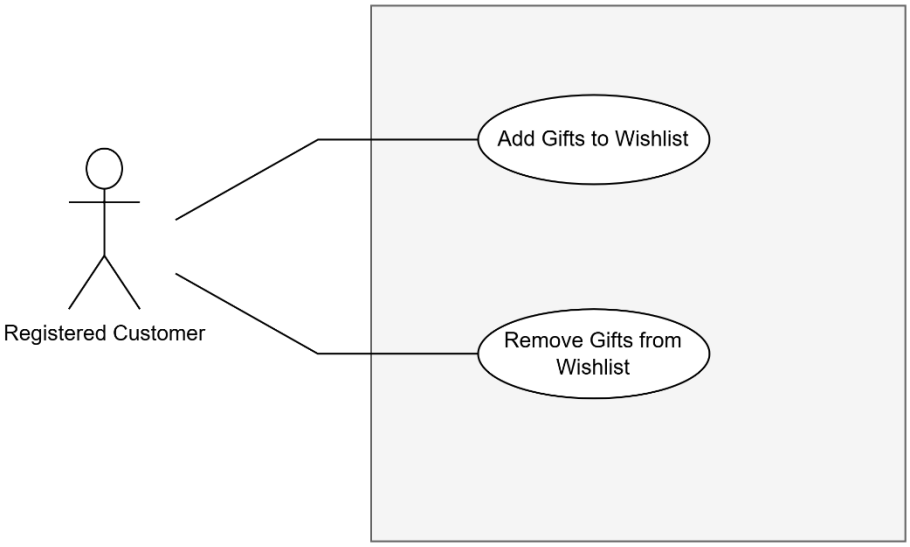
3.2.1.10 Security And Privacy:



Use Case 10 - Security And Privacy

ACTOR	Admin
PRE-CONDITION	1. The admin is authenticated and authorized to access user information. 2. The admin needs to verify the identity of a user.
POST-CONDITION	1. The updated privacy policy is published and made available to users. 2. The admin ensures that the privacy policy complies with relevant regulations and industry standards.
STEPS	1. The admin navigates to the privacy policy management page of the online bookstore system. 2. The admin reviews the current privacy policy to ensure it meets regulatory requirements and industry standards. 3. The admin identifies any areas of the privacy policy that require updating or clarification. 4. The admin works with legal and compliance teams to draft updates to the privacy policy. 5. The updated privacy policy is reviewed and approved by relevant stakeholders, such as legal, compliance, and management teams. 6. The updated privacy policy is published and made available to users, with an option for users to acknowledge their understanding and acceptance of the privacy policy. 7. The system logs all privacy policy update actions to maintain data integrity and confidentiality.

3.2.1.11 Wish list:

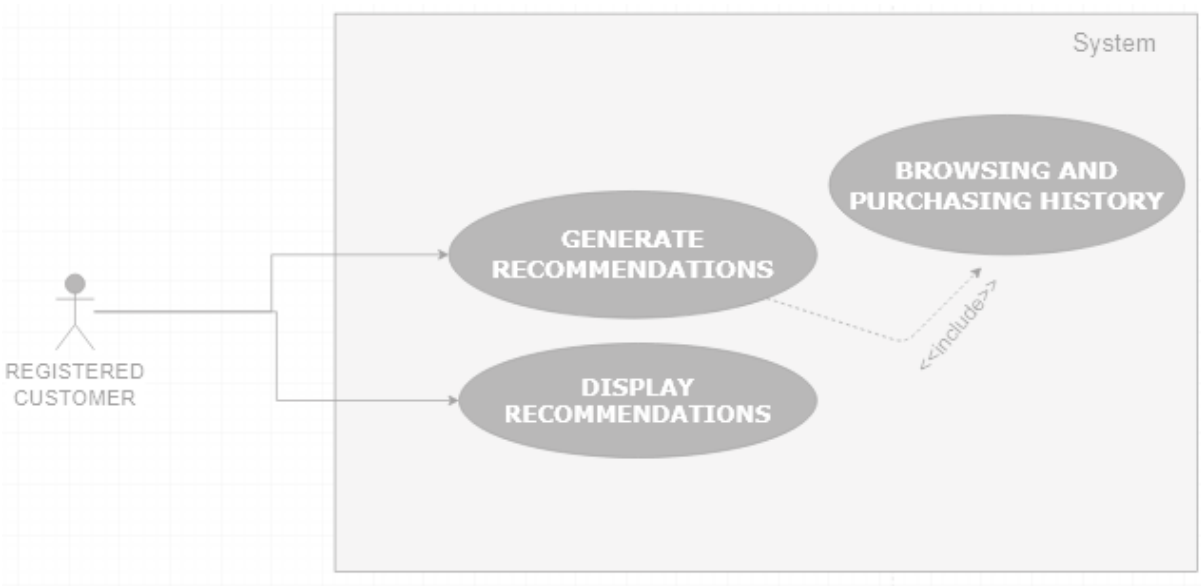


Use Case 11 - Wishlist

ACTOR	User
PRE-CONDITION	The user has accessed the Online Gift And Accessories Store's website or mobile app and has created an account.
POST-CONDITION	The user has added their desired gifts to their wish list for future reference.
STEPS	1. The user logs in to their account on the Online Gift And Accessories Store's website or mobile app. 2. The user browses through the available gifts and selects the ones they are interested in purchasing later. 3. The user selects the "Add to Wish List" button or icon next to the gift they want to save. 4. The system adds the book to the user's wish list. 5. The user can add multiple gifts to their wish list. 6. The user can view their wish list at any time

	by selecting the "Wish List" option from their account dashboard. 7. The user can remove any gift from their wish list by selecting the "Remove" button or icon next to the gift they want to remove.

3.2.1.12 Recommendations engine:

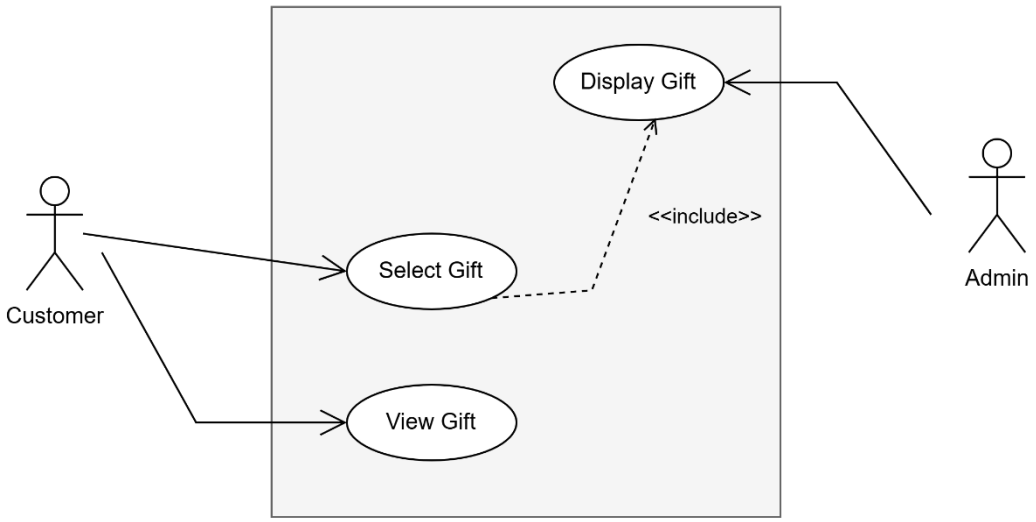


Use Case 12 - Recommendation Engine

ACTOR	User
PRE-CONDITION	The user has accessed the Online Gift And Accessories Store's website or mobile app and has created an account.
POST-CONDITION	The user has received personalized gift recommendations based on their browsing and purchase history.
STEPS	1. The user logs in to their account on the Online Gift And Accessories Store's website or mobile app. 2. The system tracks the user's browsing and purchase history, such as the genre, author, and title of the gifts they have viewed or

	bought. 3. The system applies algorithms and machine learning techniques to analyze the user's history and behavior, and generate personalized gift recommendations. 4. The system displays the recommended gifts on the user's account dashboard, homepage, or search results, with the relevant details, such as the gift cover, title, author, and rating. 5. The system also provides options for the user to filter or sort the recommendations by genre, author, rating, or price. 6. The user can view the recommended gifts, read their descriptions and reviews, and add them to their cart or wish list if they are interested. 7. The user can also provide feedback or ratings for the recommended gifts, which can further improve the recommendation engine's accuracy. 8. The system continuously updates the user's recommendation list based on their latest activity and feedback, and ensures that the recommendations are relevant, diverse, and personalized

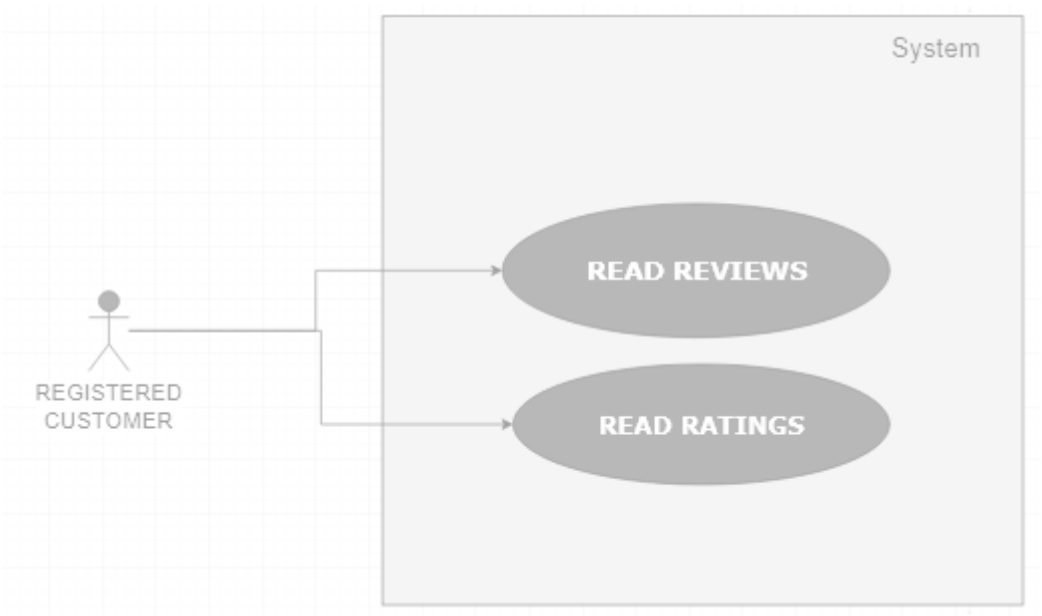
3.2.1.13 Reading Preview:



Use Case 13- Reading Preview

ACTOR	User
PRE-CONDITION	The user has accessed the Online Gift And Accessories Store's website or mobile app and has searched for a book
POST-CONDITION	The user has read a preview of the gift to help them make a purchasing decision.
STEPS	1. The user selects the gift they are interested in previewing. 2. The system displays the gift's details page, including the cover image, author bio, publisher, publication date, and user reviews. 3. The user selects the "Preview" button or icon to read a sample of the gift. 4. The system displays the preview page, which includes the first few pages or chapters of the gift. 5. The user can scroll through the preview and read the content.

3.2.1.14 Reviews and Ratings:

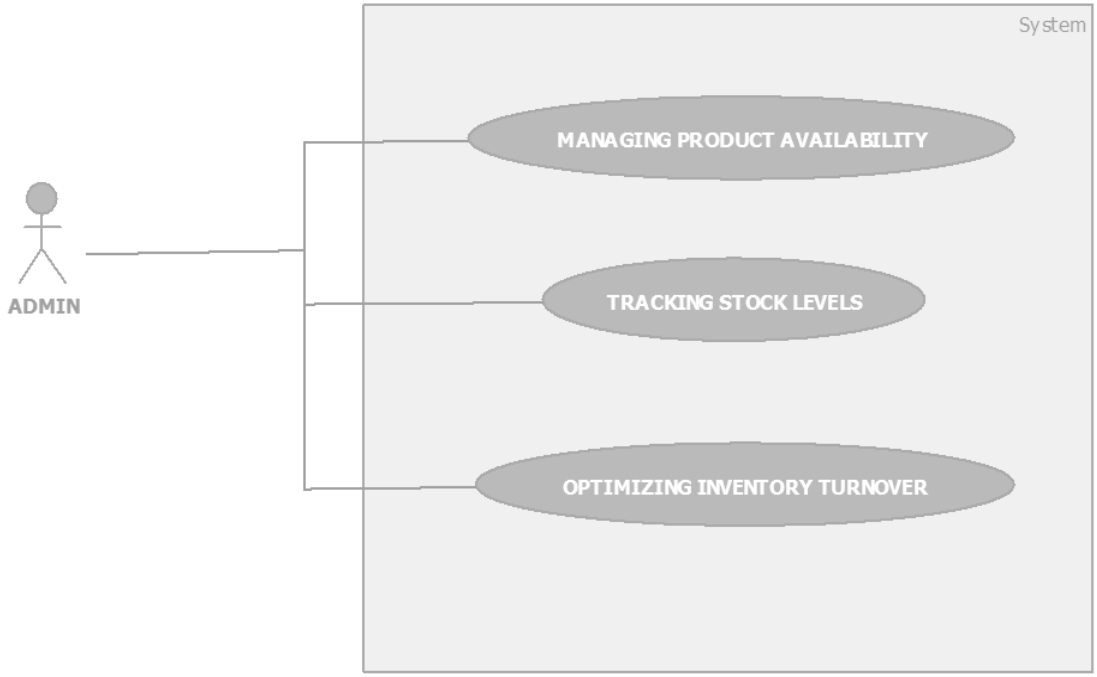


Use Case 14 - Reading Reviews

ACTOR	User
PRE-CONDITION	The user has accessed the online giftstore's website or mobile app, searched and selected a gift to review.
POST-CONDITION	The user has submitted their review and rating for the gift.
STEPS	1. The user logs in to their account on the online giftstore's website or mobile app. 2. The user searches for the gift they want to review using the search bar or by browsing the gift categories. 3. Once the user selects the gift, they are taken to the gift's page, which displays the gift's information, such as the title, author, description, cover image, and reviews and

	ratings. 4. The user scrolls down to the review and rating section of the gift's page. 5. The user selects the "Write a Review" or "Add a Rating" button. 6. The user enters their review of the gift in the text box provided. They may include their thoughts on the plot, characters, writing style, pacing, or any other aspect of the gift they want to share. 7. The user selects a star rating for the gift, usually on a scale of 1 to 5, with 1 being the lowest and 5 being the highest. 8. The user has the option to include a title for their review and their name or username. 9. The user selects the "Submit" button to post their review and rating.

3.2.1.15 Inventory Management:



Use Case 15 - Inventory Management

ACTOR	Admin
PRE-CONDITION	The admin has logged in to the Online Gift And Accessories Store's backend system and has access to the inventory management module.
POST-CONDITION	The admin has successfully managed the inventory of gifts, including adding new gifts, updating gift details, and removing gifts that are no longer available.
STEPS	1. The admin logs in to the backend system of the Online Gift And Accessories Store. 2. The admin navigates to the inventory management module and selects the "Add New Gift" option. 3. The admin enters the details of the new gift, such as author, title, publisher, publication date, ISBN, genre, and other relevant information.

	4. The admin uploads a cover image of the new gift to showcase its design. 5. The system validates the new gift details and adds it to the inventory. 6. The admin can view the list of available giftss in the inventory and their details, such as stock quantity, price, and status. 7. The admin can update the details of an existing gift, such as price, quantity, or specifications, based on customer feedback or market demand. 8. The system validates the updated gift details and updates the inventory accordingly. 9. The admin can remove a gift from the inventory that is no longer available or has been discontinued. 10. The system updates the inventory and removes the gifts from the list of available gifts. 11. The admin can generate reports and analytics on the inventory, such as stock level, sales performance, and revenue generation. 12. The admin can export the inventory data in various formats, such as CSV or Excel, for further analysis or sharing with other stakeholders.

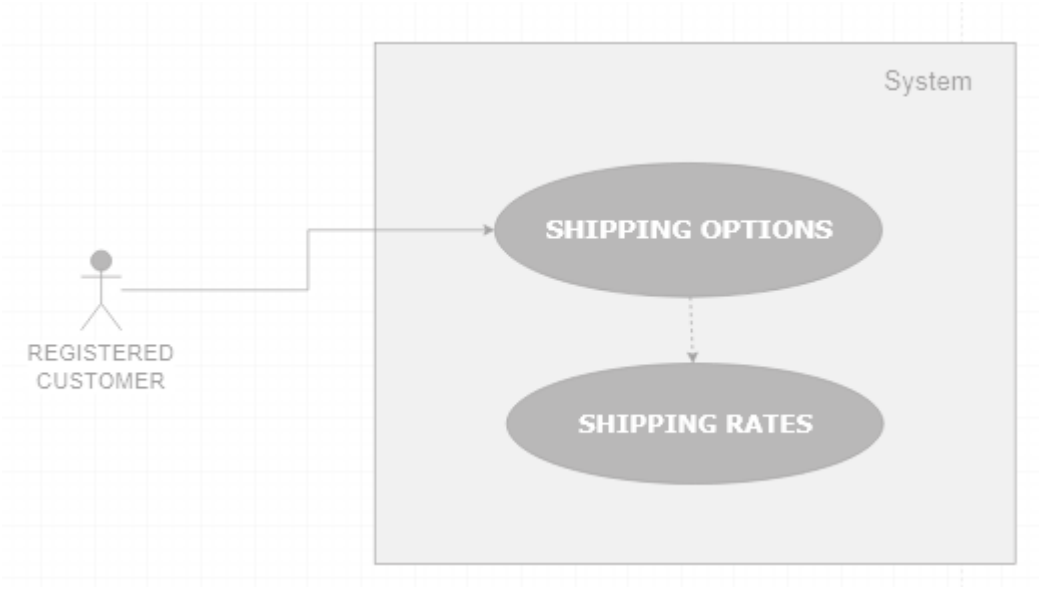
3.2.1.16 Discounts and Promotions:

Use Case 16- Discounts and Promotions

ACTOR	Admin
PRE-CONDITION	The admin has successfully logged into the admin panel.
POST-CONDITION	The admin has created, modified, or deleted a discount or promotion.
STEPS	1. Navigate to the discount and promotion management section in the admin panel. 2. Click "Create Discount" or "Create Promotion" to add a new discount or promotion, respectively. 3. Fill in the details of the discount or promotion, such as the name, discount type (e.g., percentage off, fixed amount off), discount value, start and end dates, and any eligibility criteria (e.g., minimum purchase amount). 4. Save the discount or promotion.

	5. If necessary, click "Edit" next to an existing discount or promotion to modify its details, or click "Delete" to remove it from the system. 6. Review the list of active discounts and promotions to ensure they are functioning as intended. 7. Logout from the admin panel when done.

3.2.1.17 Shipping Options:



Use Case 17 - Shipping Options

ACTOR	User
PRE-CONDITION	The user has accessed the Online Gift And Accessories Store's website or mobile app and has selected a gift for purchase.
POST-CONDITION	The user has selected a shipping option and the gift is delivered to their preferred location
STEPS	1. The user logs in to their account on the Online Gift And Accessories Store's website or mobile

	app. 2. The user selects the Gift And Accessories they want to purchase and proceeds to the checkout page. 3. The system displays the shipping options available for the user's location, such as standard, express, or same-day delivery. 4. The user can select their preferred shipping option and view the estimated delivery date and shipping cost. 5. The user can enter their shipping address or select from their saved addresses on file. 6. The user can add any additional delivery instructions, such as gate code or delivery time preference. 7. The system validates the shipping address and displays the final shipping cost and delivery date. 8. The user can proceed to the payment page and complete the transaction. 9. The system confirms the order and displays the order summary and delivery details. 10. The user can track the delivery status of the gift and receive updates on the expected delivery date.

3.2.1.18 SOCIAL MEDIA INTEGRATION:

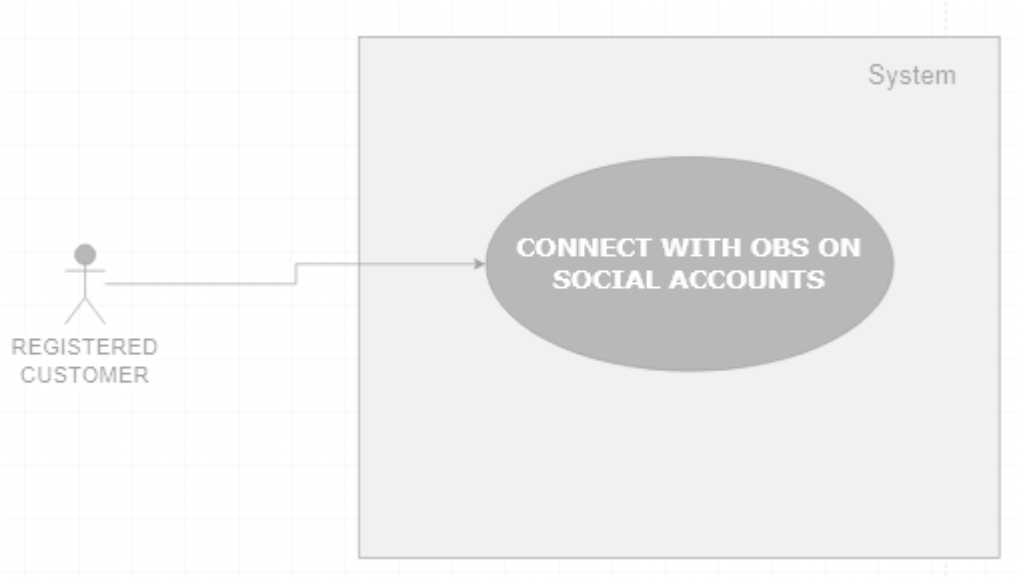


Figure 18- Social Media Integration

ACTOR	**User**
PRE-CONDITION	The user has logged in to their account on the Online Gift And Accessories Store's website.
POST-CONDITION	The user has successfully integrated their social media accounts with the Online Gift And Accessories Store's website.
STEPS	1. The user logs in to their account on the Online Gift And Accessories Store's website or mobile app. 2. The user navigates to the settings or profile section of their account. 3. The user selects the option to connect their social media accounts, such as Facebook, Twitter, or Instagram. 4. The system prompts the user to enter their social

	media account login credentials. 5. The user enters their login credentials and allows the Online Gift And Accessories Store's website or mobile app to access their social media accounts. 6. The system verifies the user's social media accounts and integrates them with the Online Gift Store's website or mobile app. 7. The user can now share their gift purchases or reviews on their social media accounts, or view and interact with content related to the Online Book Store's website or mobile app on their social media feeds. 8. The user can also receive personalized recommendations or promotions based on their social media activity and preferences.

3.2.2 Class Diagram

3.2.2.1 Account Management

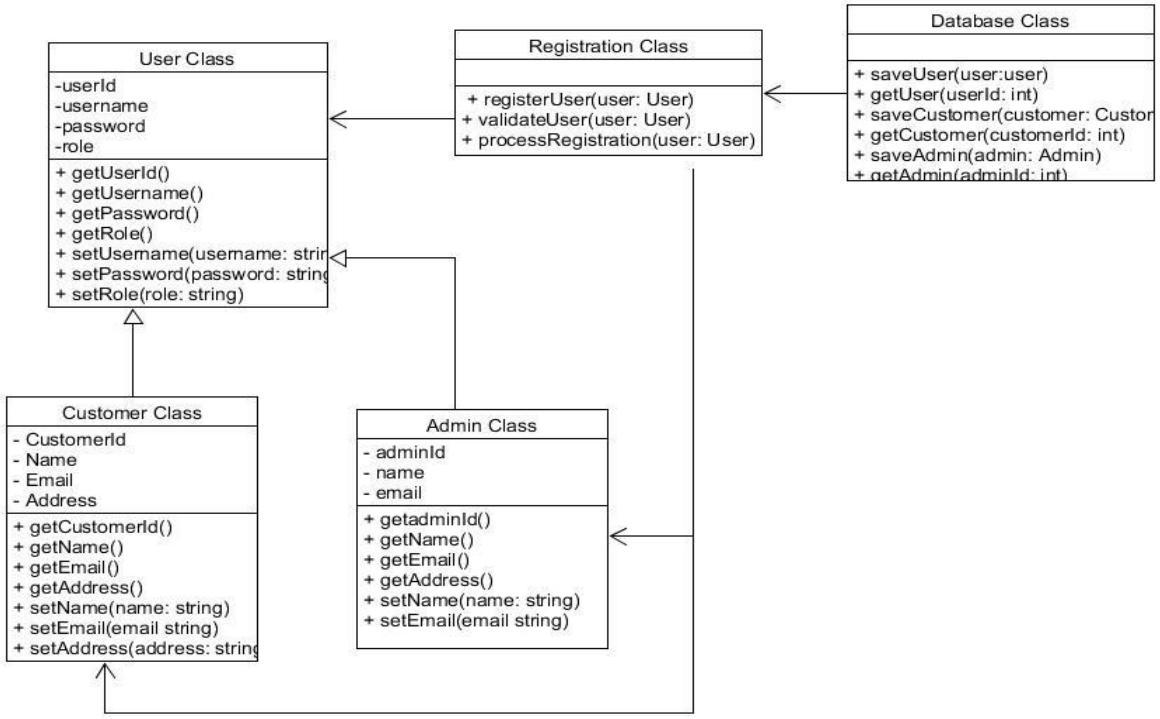


Figure 1- Account Management

3.2.2.2 Order Management

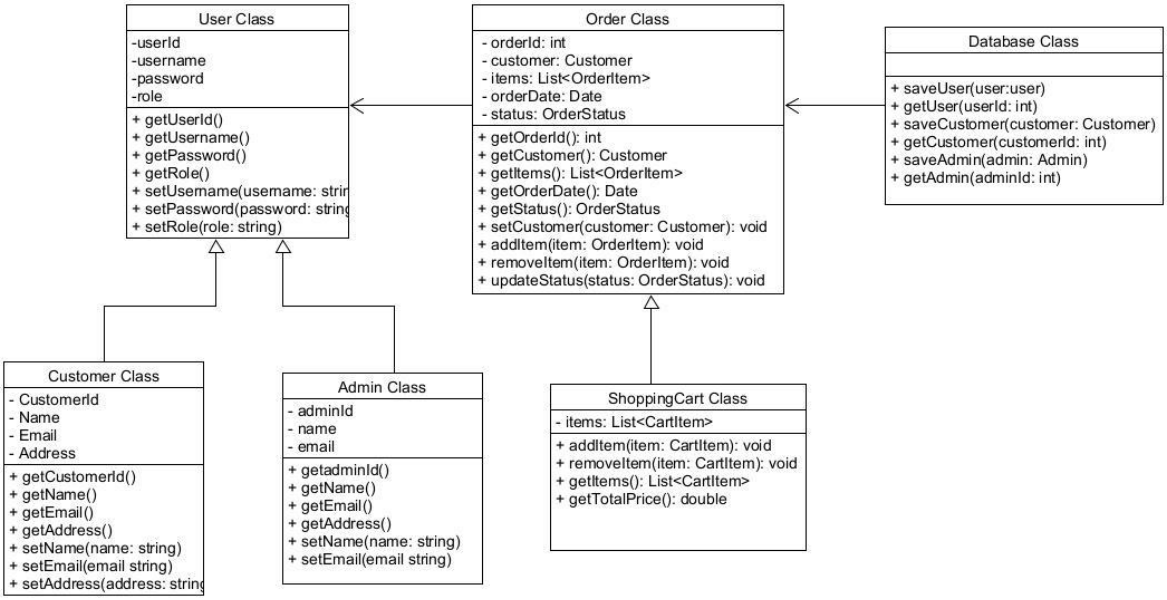


Figure 2- Order Management

3.2.2.3 Shipping Management

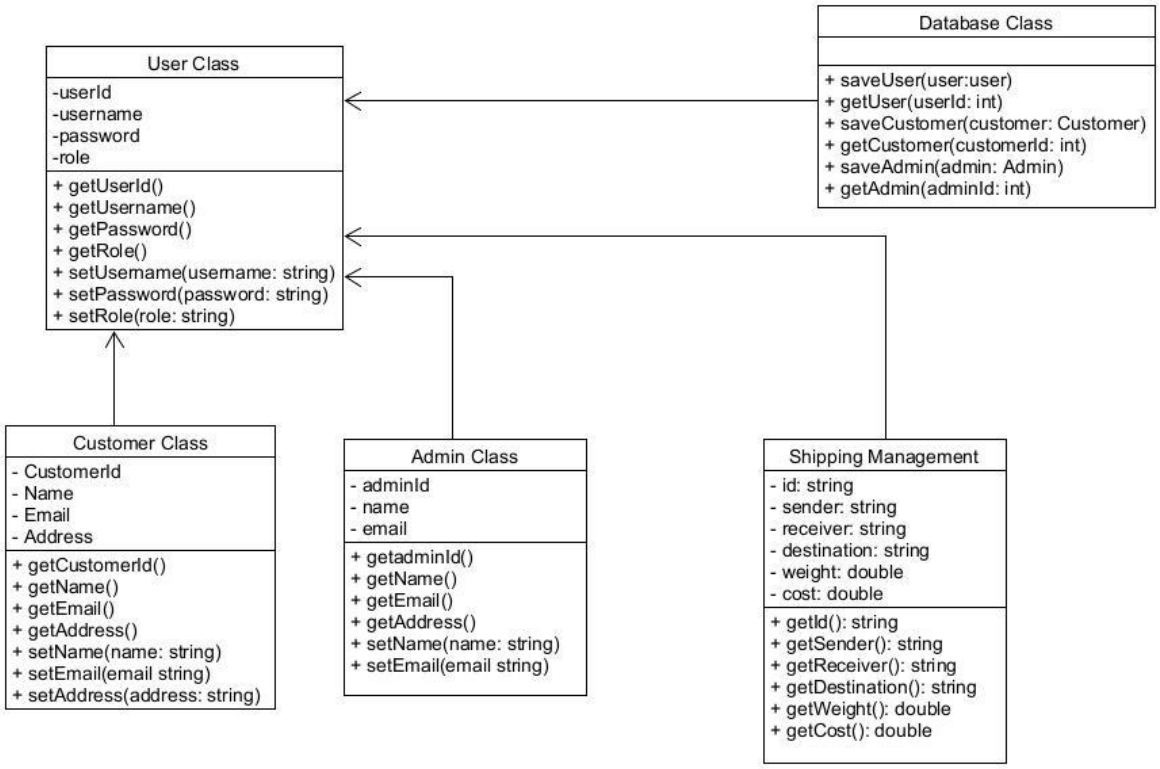


Figure 3- Shipping Management

3.2.2.4 Report Management

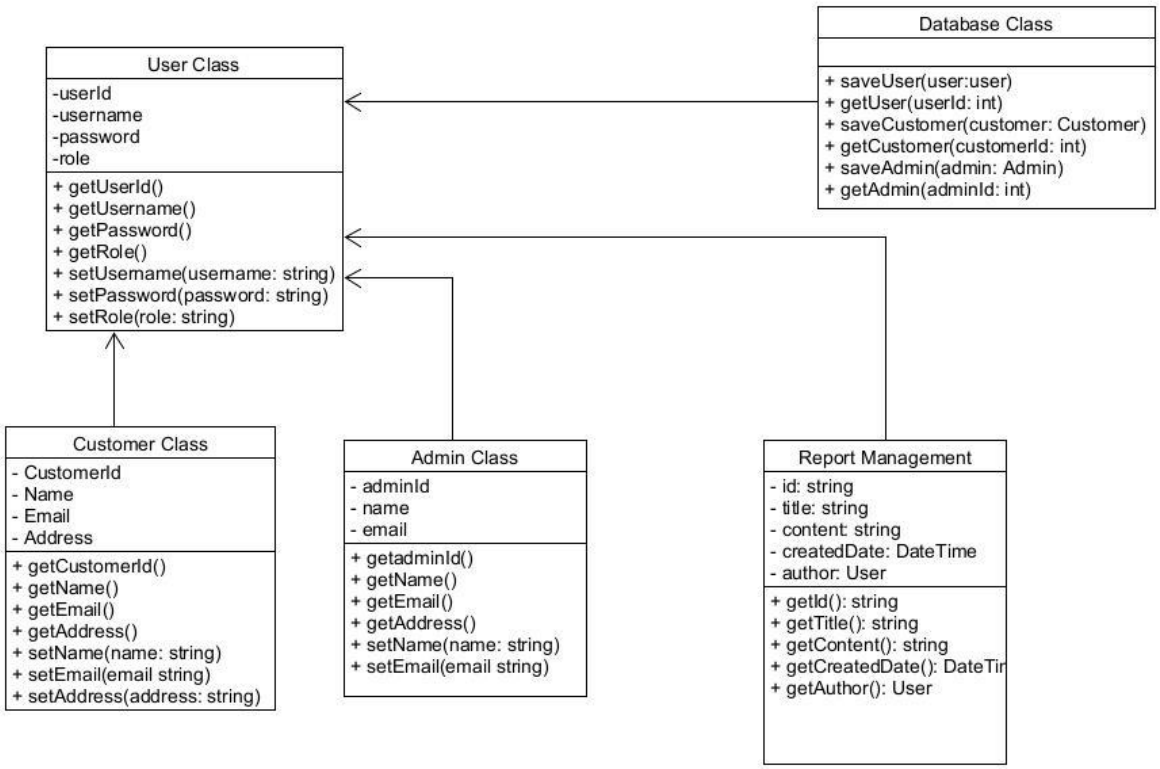


Figure 4- Report Management

3.2.2.5 Feedback Management

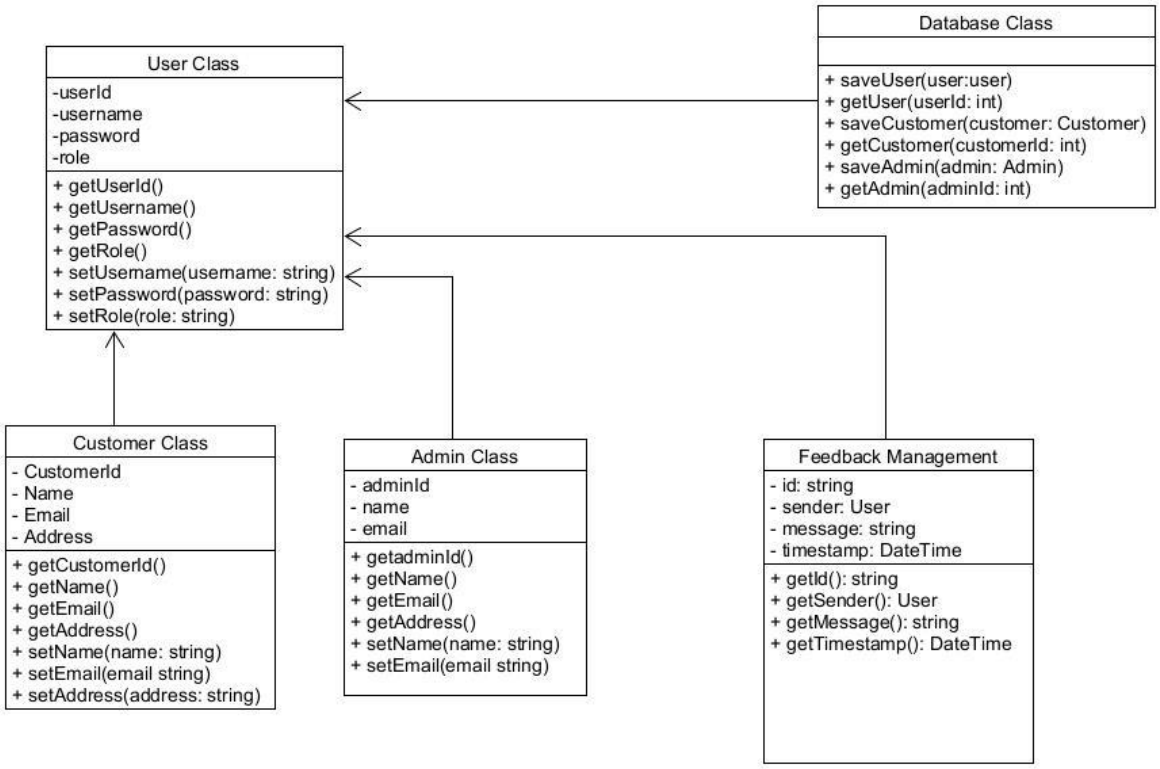
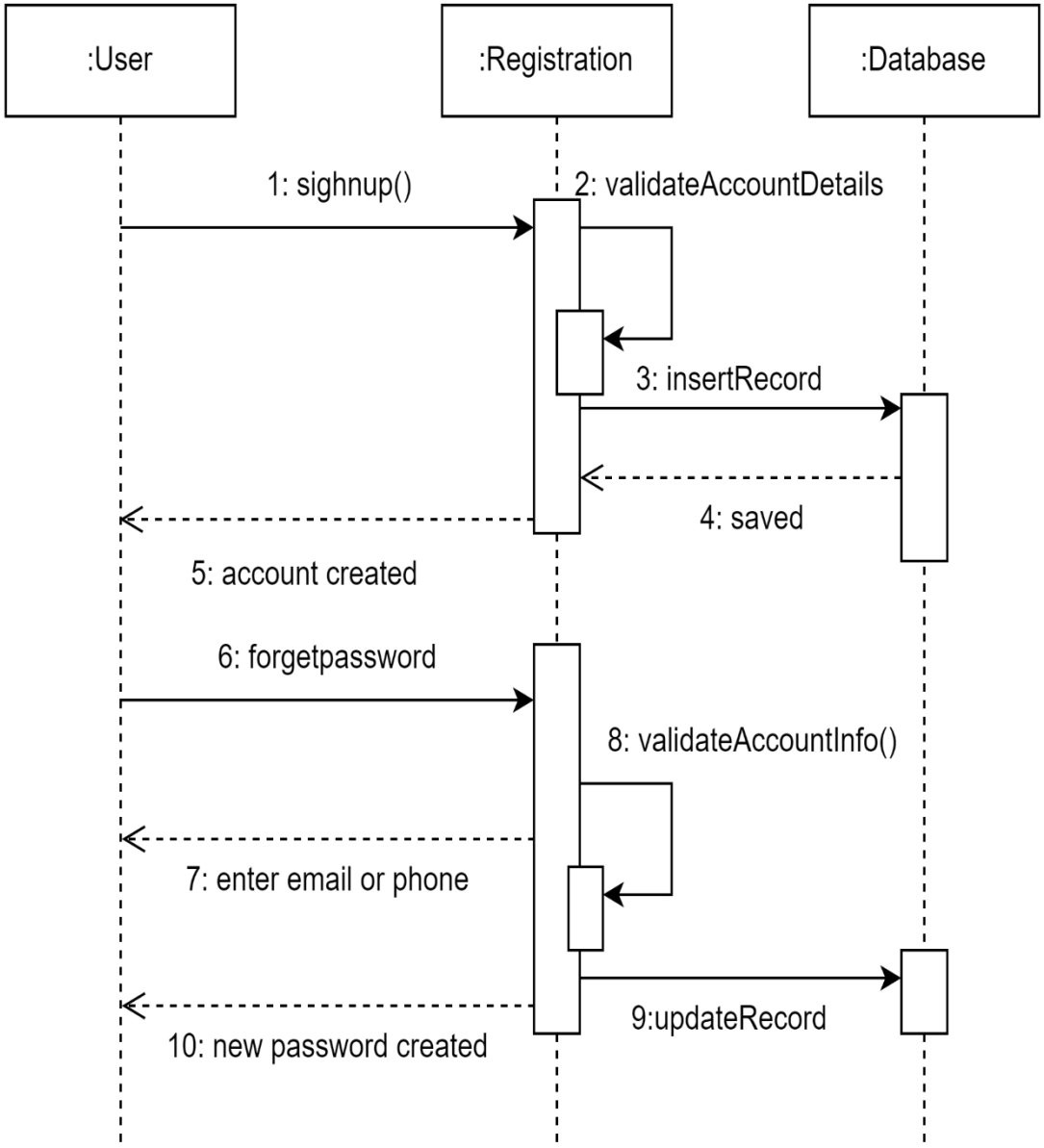


Figure 5- Feedback management

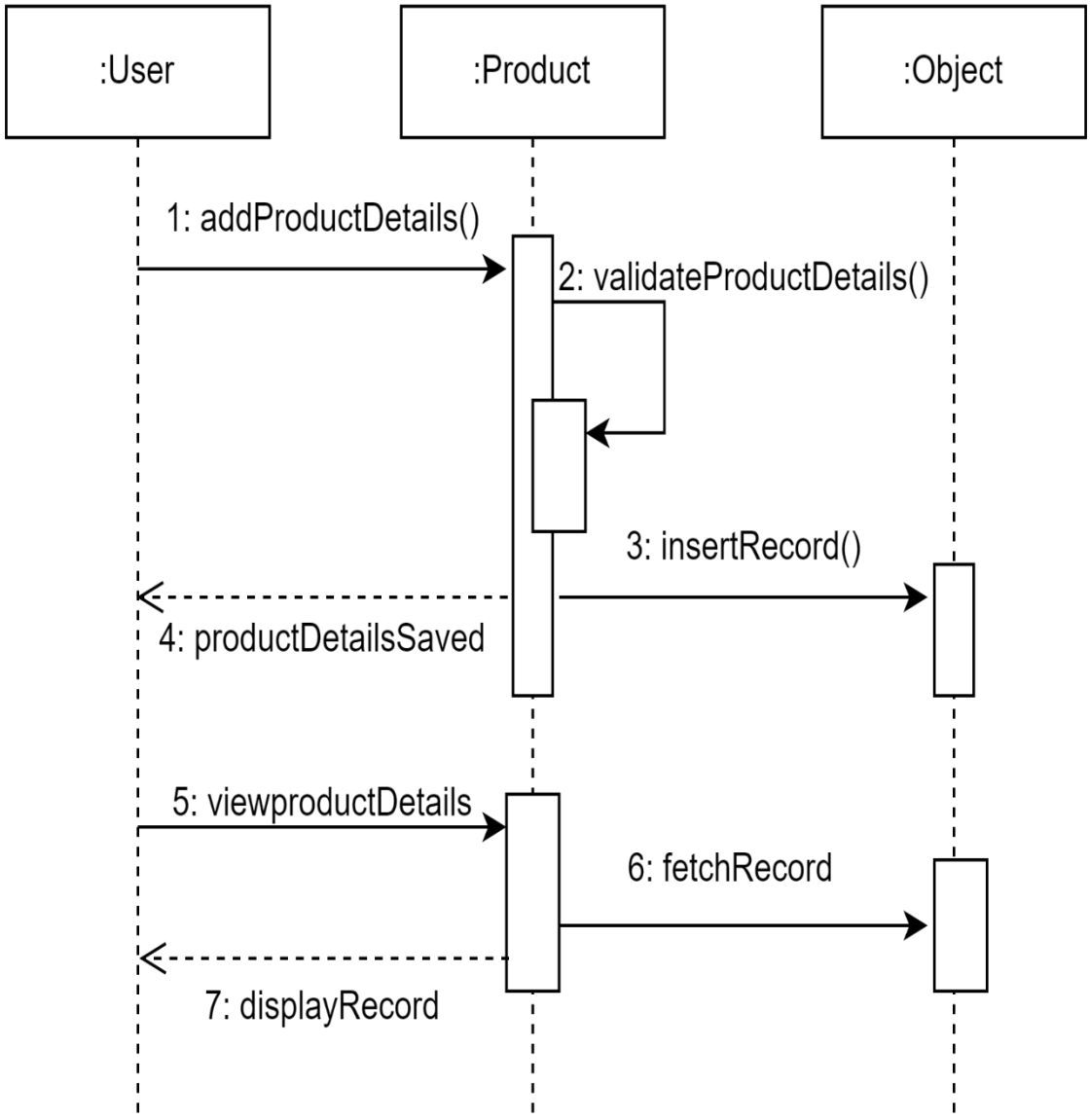
3.2.3 Sequence Diagrams:

3.2.3.1 Registration



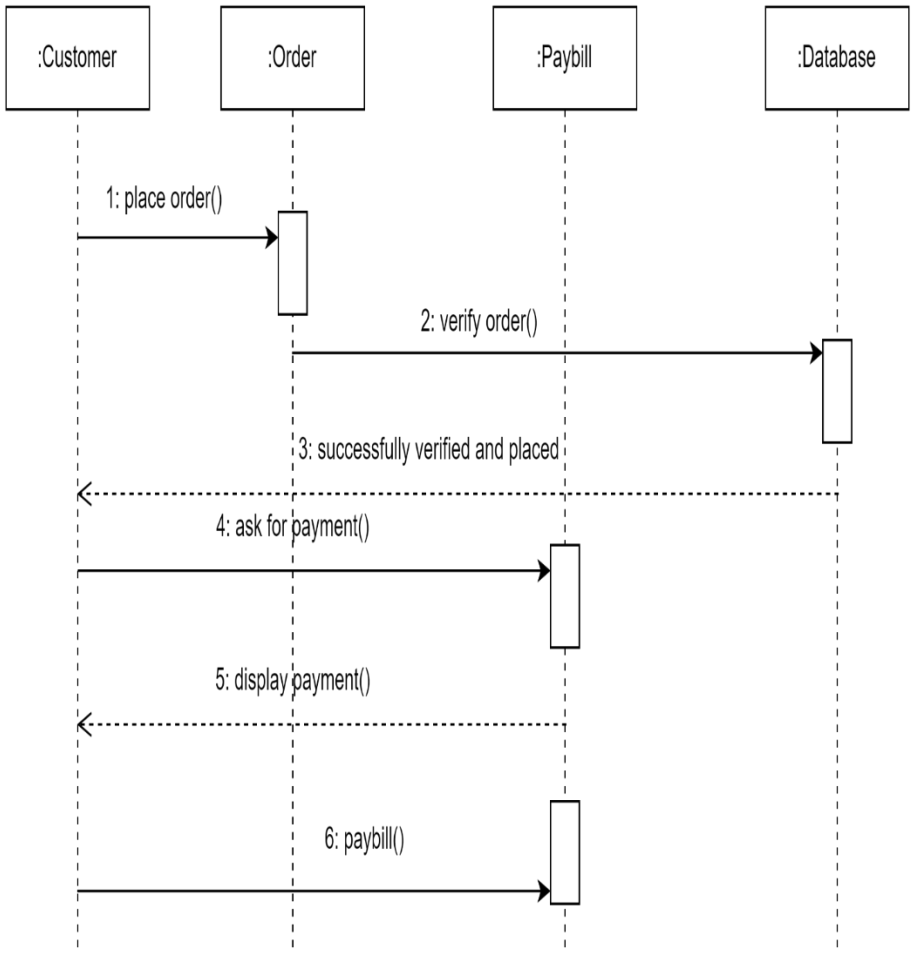
Sequence Diagram 1- Registration

3.2.3.2 Add Product



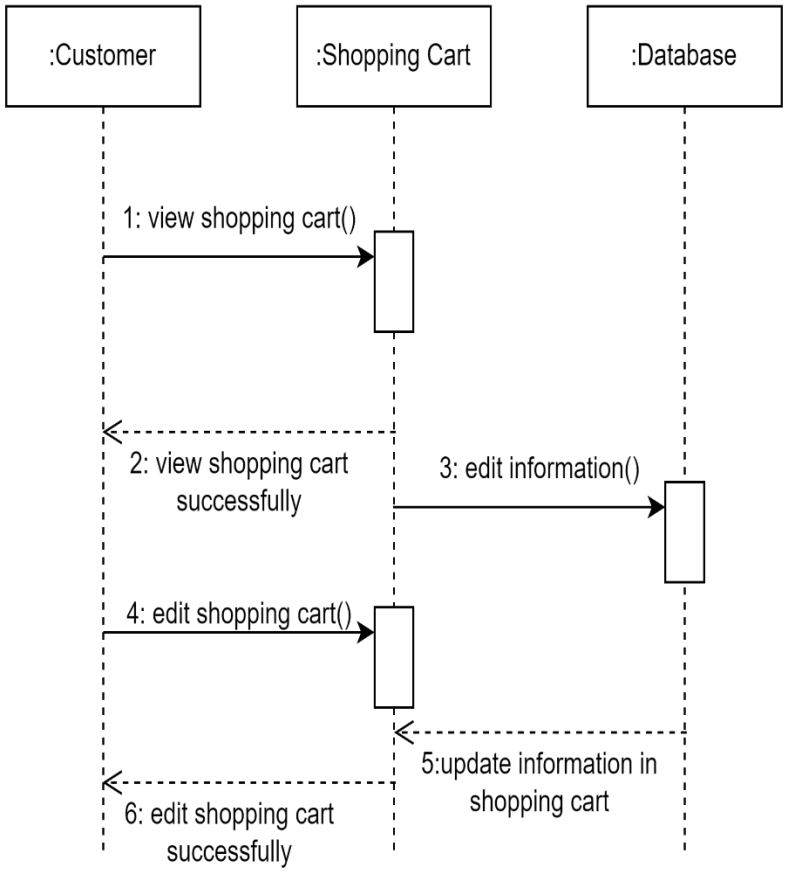
Sequence Diagram 2-Add product

3.2.3.3 Payment



Sequence Diagram 3- Payment

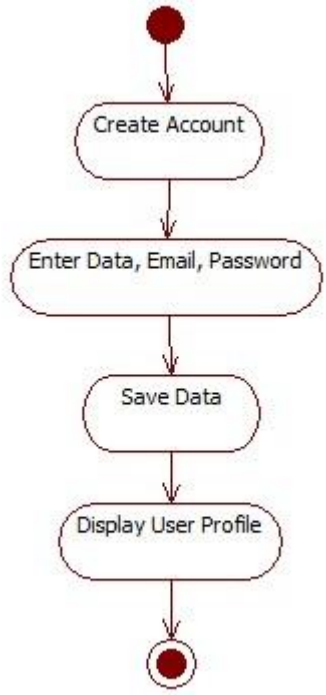
3.2.3.4 Shopping Cart



Sequence Diagram 4- Shopping Cart

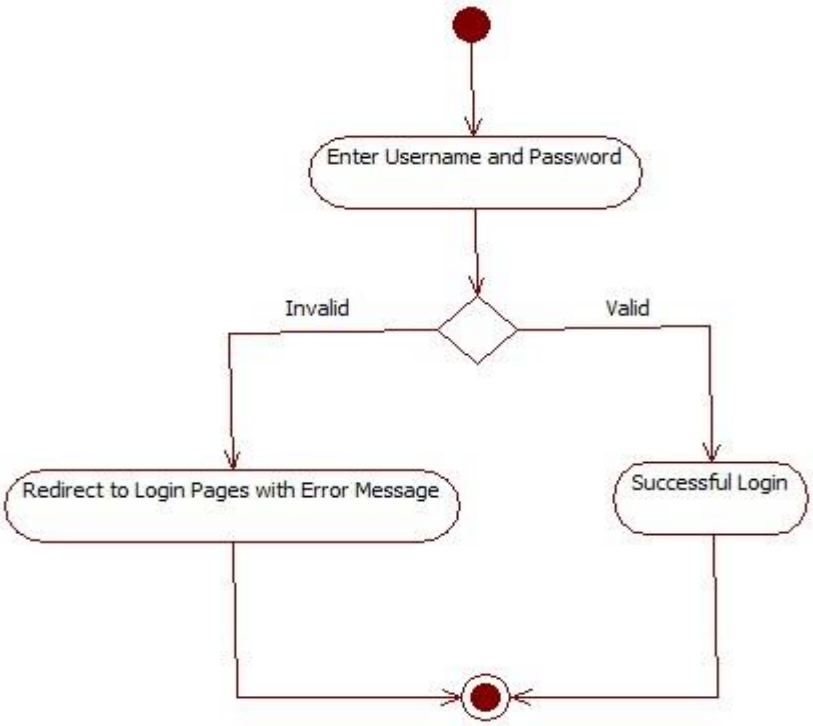
3.2.4 Activity Diagram:

3.2.4.1 Sign up



Activity Diagram 1 - Sign up

3.2.4.2 Login

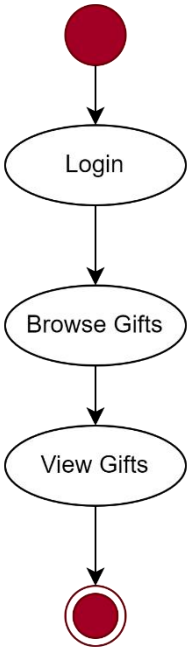


Activity Diagram 2 - Login

3.2.4.3 Shopping Cart

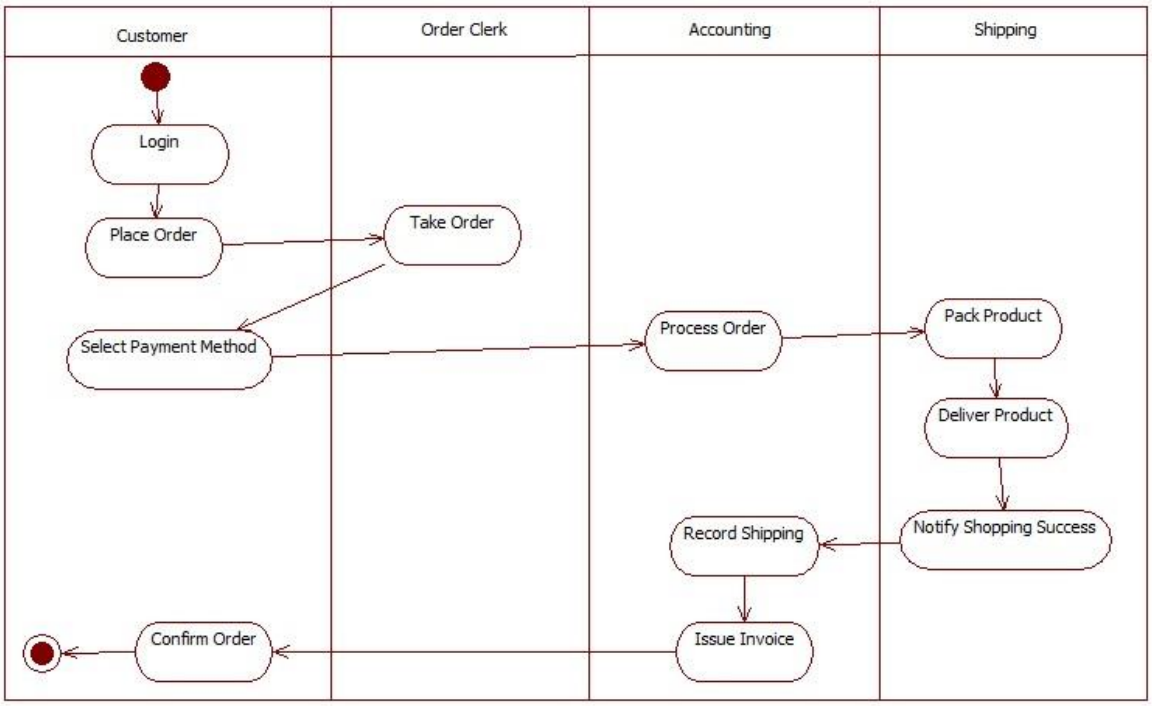
Activity Diagram 3 - Shopping Cart

3.2.4.4 View Product



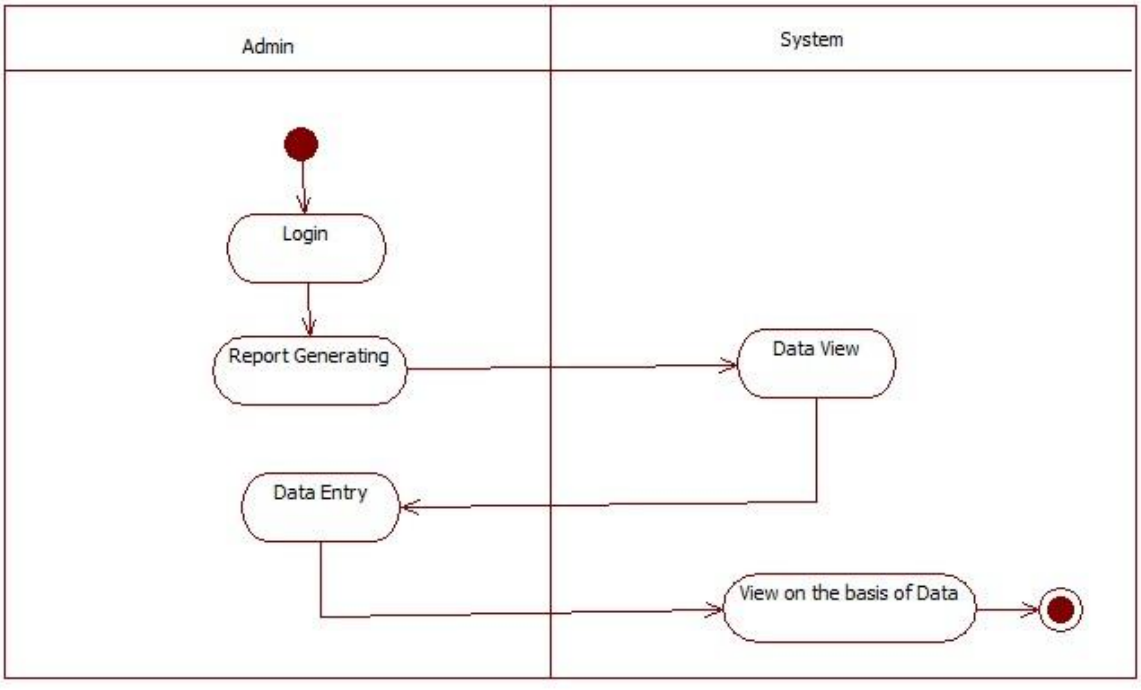
Activity Diagram 4 - View Product

3.2.4.5 Place Order and Shipment



Activity Diagram 5 - Place Order and Shipment

3.2.4.6 Reports Generation



Activity Diagram 6- Report Generation

3.3 Design Rationale:

The design rationale for an Online Gift And Accessories Store involves creating a user-friendly platform with a seamless user experience. It includes intuitive navigation, responsive design, and efficient search and filtering options. The design emphasizes comprehensive gift details, customer reviews, and a secure shopping cart and checkout process. Personalized recommendations enhance user engagement, while security measures and trust-building elements instill confidence. Integration with inventory management ensures accurate product availability. Feedback channels and customer support are provided for assistance. Scalability and performance optimizations prepare the platform for growth.

4 Data Design

4.1 Data Description:

MySQL database connects by using PHP Scripts installed on the local web server. The PHP script establishes a connection to the MySQL database. Once the connection is established, you can execute SQL queries on the database. After executing the required SQL queries, it's important to close the database connection.

You can build upon this foundation to perform various operations such as inserting, updating, and deleting data as per your requirements.

4.1.1 Data Objects:

4.1.1.1 Users:

userName	The name of the user.
userPassword	The password of the user.
userEmail	The email of User.
userPhone	The phone number of user.
userAddress	The address of user.
userGender	The gender of user.

4.1.1.2 Login:

userName	The name of the user.
userPassword	The password of the user.

4.1.1.3 Gifts:

Title	The title of the gifts.
Author	The author(s) of the gifts.
ISBN	The International Standard Gift Number assigned to the gift.
Description	A brief description of the gift.
Price	The price of the gift.
Category	The Category or genre of Gift.
Publication_Date	The date when the book was published.
Publisher	The publishing company of the Gift.
Availability	The availability status of the gift (in stock, out of stock, etc.).

4.1.1.4 Roles:

roleName	The name of the role.
roleDescription	The description of the role

4.1.1.5 Payments:

payDate	The date of the Payment.
payDescription	The description of payment
payType	The type of payment

4.1.1.6 Bookings:

bookingDate	The date of Booking.
bookingDescription	The description of Booking

4.1.1.7 Permission:

permissionName	The name of Permission
permissionModule	The Module of Permission.

4.1.1.8 Categories:

categoriesName	The name of Categories.

4.1.2 ER Diagram:

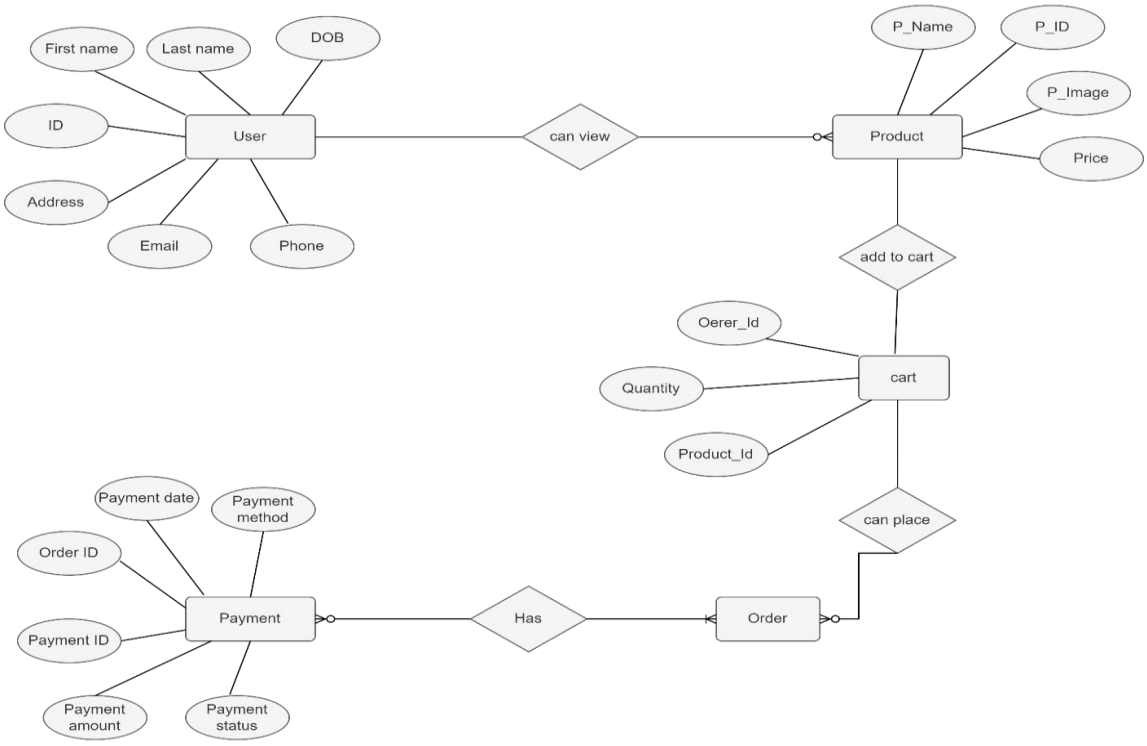


Figure 2 - Entity Relationship Diagram

4.2 Data Dictionary:

4.2.1.1 Users:

userID	Int
userName	String
userPassword	String
userEmail	String
userPhone	Int
userAddress	string
userGender	string

4.2.1.2 Login:

loginID	int
loginRoleID	int
userName	string
userPassword	string

4.2.1.3 Gifts:

giftID	int
Title	string
Author	string
ISBN	int
Description	string
Price	int
CategoryId	int
Publication_Date	date
Publisher	string
Availability	boolean

4.2.1.4 Roles:

roleID	int
roleName	string
roleDescription	string

4.2.1.5 Payments:

payID	int
payDate	date
payDescription	string
payType	string

4.2.1.6 Bookings:

bookingID	int
bookingDate	date
bookingDescription	String

4.2.1.7 Permission:

permissionID	Int
permissionName	String
permissionModule	String

4.2.1.8 Categories:

categoryID	Int
categoriesName	String

5 Component Design:

5.1 Registration:

Class Name	**Registration**
Attributes	• username (string): Stores the username chosen by the user during registration. • password (string): Stores the password chosen by the user. • email (string): Stores the email address provided by the user.
Methods	1. registerUser() : boolean • Performs the registration process by calling the necessary validation methods and storing the user's information. • Returns true if the registration is successful; otherwise, returns false. 2. validateUsername() : boolean • Validates the username to ensure it meets the required criteria (e.g., length, uniqueness). • Returns true if the username is valid; otherwise, returns false.

	3. validatePassword() : boolean • Validates the password to ensure it meets the required criteria (e.g., length, complexity). • Returns true if the password is valid; otherwise, returns false. 4. validateEmail() : boolean • Validates the email address to ensure it is in the correct format. • Returns true if the email is valid; otherwise, returns false. 5. Process registration: boolean • The processRegistration() method takes the username, password, and email as input parameters. It sets these values using the respective setter methods (setUsername(), setPassword(), setEmail()). • If any of the validation checks fail, it returns false to indicate the failure.

Pseudo Code	class Registration {
	// Attributes
	username
	password
	email
	// Methods
	processRegistration(username, password, email) {
	this.setUsername(username)
	this.setPassword(password)
	this.setEmail(email)
	if (this.registerUser()) {
	// Successful registration
	return true
	} else {
	// Handle registration failure
	return false
	}
	}

	validateUsername() {
	// Validation logic
	// Return true or false based on the validation result
	}
	validatePassword() {
	// Validation logic
	// Return true or false based on the validation result
	}
	validateEmail() {
	// Validation logic
	// Return true or false based on the validation result
	}
	registerUser() {
	if (!this.validateUsername()) {
	return false
	}
	if (!this.validatePassword()) {
	return false
	}
	if (!this.validateEmail()) {
	return false
	}
	return true
	}
	}

5.2 User Class:

Class Name	**User**
Attributes	• username (string): Stores the username chosen by the user during registration. • password (string): Stores the password chosen by the

	user. • email (string): Stores the email address provided by the user.
Methods	1. setUsername(username: string) • Sets the username attribute to the provided value. 2. setPassword(password: string) • Sets the password attribute to the provided value. 3. setEmail(email: string) • Sets the email attribute to the provided value. 4. getUsername() : string • Returns the username of the user. 5. getPassword() : string • Returns the password of the user. 6. getEmail() : string • Returns the email address of the user.

Pseudo Code	class User {
	// Attributes
	username
	password
	email
	// Methods
	setUsername(username) {
	this.username = username
	}
	setPassword(password) {
	this.password = password
	}
	setEmail(email) {
	this.email = email
	}
	getUsername() {
	return this.username
	}
	getPassword() {

	return this.password
	}
	getEmail() {
	return this.email
	}
	}

5.3 Admin Class

Class Name	**Admin**
Attributes	• username (string): Stores the username of the admin. • password (string): Stores the password of the admin. • email (string): Stores the email address of the admin. • role (string): Stores the role of the admin.
Methods	1. setUsername(username: string) • Sets the username attribute to the provided value. 2. setPassword(password: string) • Sets the password attribute to the provided value. 3. setEmail(email: string) • Sets the email attribute to the provided value. 4. setRole(role: string) • Sets the role attribute to the provided value. 5. getUsername() : string • Returns the username of the admin. 6. getPassword() : string • Returns the password of the admin. 7. getEmail() : string • Returns the email address of the admin. 8. getRole() : string • Returns the role of the admin.

Pseudo Code	class Admin {
	// Attributes
	username
	password
	email
	role
	// Methods

	setUsername(username) {
	this.username = username
	}
	setPassword(password) {
	this.password = password
	}
	setEmail(email) {
	this.email = email
	}
	setRole(role) {
	this.role = role
	}
	getUsername() {
	return this.username
	}
	getPassword() {
	return this.password
	}
	getEmail() {
	return this.email
	}
	getRole() {
	return this.role
	}
	}

5.4 Product Management:

Class Name	**Product**
Attributes	• productId (string): Stores the unique identifier of the product. • name (string): Stores the name of the product.

	• price (float): Stores the price of the product. • description (string): Stores the description of the product. • quantity (int): Stores the quantity of the product available in stock.
Methods	1. setProductId(productId: string) • Sets the productId attribute to the provided value. 2. setName(name: string) • Sets the name attribute to the provided value. 3. setPrice(price: float) • Sets the price attribute to the provided value. 4. setDescription(description: string) • Sets the description attribute to the provided value. 5. setQuantity(quantity: int) • Sets the quantity attribute to the provided value. 6. getProductId() : string • Returns the productId of the product. 7. getName() : string • Returns the name of the product. 8. getPrice() : float • Returns the price of the product. 9. getDescription() : string • Returns the description of the product. 10. getQuantity() : int • Returns the quantity of the product.

Pseudo Code	class User {
	// Attributes
	username
	password
	email
	// Methods
	setUsername(username) {
	this.username = username
	}
	setPassword(password) {
	this.password = password
	}

	setEmail(email) {
	this.email = email
	}
	getUsername() {
	return this.username
	}
	getPassword() {
	return this.password
	}
	getEmail() {
	return this.email
	}
	}

5.5 Shopping Cart Management:

Class Name	**Shopping Cart**
Attributes	• cartItems (list): Stores the items added to the shopping cart.
Methods	1. addItem(item: Item) • Adds an item to the shopping cart. • Takes an Item object as a parameter. • Adds the item to the cartItems list. 2. removeItem(item: Item) • Removes an item from the shopping cart. • Takes an Item object as a parameter. • Removes the item from the cartItems list. 3. getCartItems() : list • Returns the list of items in the shopping cart. 4. getTotalPrice() : float • Calculates and returns the total price of all items in the shopping cart. 5. clearCart() • Clears the shopping cart by emptying the cartItems list.

Pseudo Code	class ShoppingCart {
	// Attributes
	cartItems
	// Methods
	addItem(item) {
	cartItems.add(item)
	}
	removeItem(item) {
	cartItems.remove(item)
	}
	getCartItems() {
	return cartItems
	}
	getTotalPrice() {
	total = 0
	for item in cartItems:
	total += item.getPrice()
	return total
	}
	clearCart() {
	cartItems.clear()
	}
	}

5.6 Order Management:

Class Name	**Order**
Attributes	• orderId (string): Stores the unique identifier of the order. • customer (Customer): Stores the customer associated with the order. • items (list): Stores the items included in the order.

	• totalAmount (float): Stores the total amount of the order. • status (string): Stores the status of the order (e.g., "pending", "shipped").
Methods	1. setOrderId(orderId: string) • Sets the orderId attribute to the provided value. 2. setCustomer(customer: Customer) • Sets the customer attribute to the provided Customer object. 3. addItem(item: Item) • Adds an item to the order. • Takes an Item object as a parameter. • Adds the item to the items list. 4. removeItem(item: Item) • Removes an item from the order. • Takes an Item object as a parameter. • Removes the item from the items list. 5. calculateTotalAmount() • Calculates the total amount of the order by summing up the prices of all items. • Updates the totalAmount attribute with the calculated value. 6. setStatus(status: string) • Sets the status attribute to the provided value. 7. getOrderId() : string • Returns the orderId of the order. 8. getCustomer() : Customer • Returns the customer associated with the order. 9. getItems() : list • Returns the list of items in the order. 10. getTotalAmount() : float • Returns the total amount of the order. 11. getStatus() : string • Returns the status of the order.

Pseudo Code	class Order {
	// Attributes
	orderId
	customer
	items
	totalAmount

	status
	// Methods
	setOrderId(orderId) {
	this.orderId = orderId
	}
	setCustomer(customer) {
	this.customer = customer
	}
	addItem(item) {
	items.add(item)
	}
	removeItem(item) {
	items.remove(item)
	}
	calculateTotalAmount() {
	totalAmount = 0
	for item in items:
	totalAmount += item.getPrice()
	}
	setStatus(status) {
	this.status = status
	}
	getOrderId() {
	return orderId
	}
	getCustomer() {
	return customer
	}
	getItems() {
	return items

	}
	getTotalAmount() {
	return totalAmount
	}
	getStatus() {
	return status
	}
	}

5.7 Shipment Management

Class Name	**Shipment**
Attributes	• shipmentId (string): Stores the unique identifier of the shipment. • orderId (string): Stores the identifier of the order associated with the shipment. • shippingAddress (string): Stores the address where the shipment is to be delivered. • status (string): Stores the status of the shipment (e.g., "pending", "in transit", "delivered").
Methods	1. setShipmentId(shipmentId: string) • Sets the shipmentId attribute to the provided value. 2. setOrderId(orderId: string) • Sets the orderId attribute to the provided value. 3. setShippingAddress(shippingAddress: string) • Sets the shippingAddress attribute to the provided value. 4. setStatus(status: string) • Sets the status attribute to the provided value. 5. getShipmentId() : string • Returns the shipmentId of the shipment. 6. getOrderId() : string • Returns the orderId associated with the shipment. 7. getShippingAddress() : string • Returns the shipping address of the shipment. 8. getStatus() : string • Returns the status of the shipment.
Pseudo Code	class Shipment {

	// Attributes
	shipmentId
	orderId
	shippingAddress
	status
	// Methods
	setShipmentId(shipmentId) {
	this.shipmentId = shipmentId
	}
	setOrderId(orderId) {
	this.orderId = orderId
	}
	setShippingAddress(shippingAddress) {
	this.shippingAddress = shippingAddress
	}
	setStatus(status) {
	this.status = status
	}
	getShipmentId() {
	return shipmentId
	}
	getOrderId() {
	return orderId
	}
	getShippingAddress() {
	return shippingAddress
	}
	getStatus() {
	return status
	}

	}

5.8 Report Management

Class Name	**Report**
Attributes	• reports (list): Stores the reports managed by the system.
Methods	1. addReport(report: Report) • Adds a report to the system. • Takes a Report object as a parameter. • Adds the report to the reports list. 2. removeReport(report: Report) • Removes a report from the system. • Takes a Report object as a parameter. • Removes the report from the reports list. 3. generateReport(reportType: string) • Generates a specific type of report. • Takes a reportType as a parameter. • Implements the logic to generate the specified report. 4. viewAllReports() : list • Returns a list of all the reports in the system. 5. getReportById(reportId: string) : Report • Returns a report object based on the provided reportId.

Pseudo Code	class ReportManagement {
	// Attributes
	reports
	// Methods
	addReport(report) {
	reports.add(report)
	}
	removeReport(report) {
	reports.remove(report)
	}
	generateReport(reportType) {
	// Logic to generate the specified report

	}
	viewAllReports() {
	return reports
	}
	getReportById(reportId) {
	for report in reports:
	if report.getId() == reportId:
	return report
	return null // If report is not found
	}
	}

5.9 Feedback Management

Class Name	**Feedback**
Attributes	• feedbackId (string): Stores the unique identifier of the feedback. • customerName (string): Stores the name of the customer who provided the feedback. • rating (int): Stores the rating given by the customer (e.g., on a scale of 1 to 5). • comment (string): Stores the comment or feedback message provided by the customer.
Methods	1. setFeedbackId(feedbackId: string) • Sets the feedbackId attribute to the provided value. 2. setCustomerName(customerName: string) • Sets the customerName attribute to the provided value. 3. setRating(rating: int) • Sets the rating attribute to the provided value. 4. setComment(comment: string) • Sets the comment attribute to the provided value. 5. getFeedbackId() : string • Returns the feedbackId of the feedback. 6. getCustomerName() : string • Returns the name of the customer who provided the

	feedback. 7. getRating() : int • Returns the rating given by the customer. 8. getComment() : string • Returns the comment or feedback message provided by the customer.

Pseudo Code	class Feedback {
	// Attributes
	feedbackId
	customerName
	rating
	comment
	// Methods
	setFeedbackId(feedbackId) {
	this.feedbackId = feedbackId
	}
	setCustomerName(customerName) {
	this.customerName = customerName
	}
	setRating(rating) {
	this.rating = rating
	}
	setComment(comment) {
	this.comment = comment
	}
	getFeedbackId() {
	return feedbackId
	}
	getCustomerName() {
	return customerName
	}
	getRating() {

	return rating
	}
	getComment() {
	return comment
	}
	}

6 Human Interface Design:

Human interface design (also known as user interface design) is an important aspect of designing an Online Gift And Accessories Store. It focuses on creating intuitive, user-friendly interfaces that facilitate easy navigation, efficient task completion, and a pleasant user experience.

6.1 Overview: Functionality of System from User's Perspective:

From a user perspective, an Online Gift And Accessories Store should provide a comprehensive set of functionalities to ensure a smooth and satisfying experience. Here's an overview of the key functionalities that users would expect from an Online Gift And Accessories Store:

6.1.1 Browsing and Search:

- Users should be able to browse through a vast collection of gifts, categorized by genres, authors, bestsellers, or other relevant filters.
- A search functionality should allow users to find specific gifts based on keywords, titles, authors, or ISBN.

6.1.2 Gift Listings and Details:

- Each gift listing should display key information such as gift cover image, title, author, price, available formats (hardcover, paperback, ebook), and customer reviews/ratings.

- Users should be able to access detailed gift descriptions, including a summary.

6.1.3 Virtual Gift Preview:

- Provide a virtual gift preview feature that allows users to preview a portion of the gift's content before making a purchase decision.
- Implement a gift image flip feature or interactive user to simulate the experience of flipping through pages.

6.1.4 Shopping Cart and Wishlist:

- Users should be able to add gifts to their shopping cart for future purchase and manage the quantities of selected gifts.
- Enable users to create wishlists to save gifts they are interested in for future reference or purchase.

6.1.5 User Reviews and Recommendations:

- Allow users to read and submit reviews and ratings for gifts they have seen or purchased.
- Provide personalized gift recommendations based on users' browsing and purchase history.

6.1.6 Account Management:

- Enable users to create accounts, log in, and manage their personal information.
- Store users' information for easy access during the checkout process and to provide personalized recommendations.

6.1.7 Online Ordering and Checkout:

- Provide a seamless and secure online ordering process, allowing users to review their shopping cart, select shipping options, and proceed to checkout.
- Incorporate multiple payment options, such as credit cards, PayPal, or other popular online payment methods.
- Clearly communicate shipping details, estimated delivery times, and order tracking information.

6.1.8 Customer Support and Assistance:

- Offer various channels for customer support, such as live chat, email, or phone, to address user queries, provide assistance with orders, or handle returns and refunds.

- Provide an FAQ section or knowledge base with answers to common questions about ordering, shipping, returns, and other book-related inquiries.

6.1.9 Social Sharing and Integration:

- Integrate social media sharing buttons to allow users to share their favorite gifts or purchases on their social profiles.
- Connect the Online Gift And Accessories Store with social media platforms to engage with users, announce new releases, and showcase customer reviews and recommendations.

7 Requirement Matrix:

Components: Requirements from SRS (Use case)	Account Management	Product Management	Shopping Cart Management	Order Management	Shipping Management	Report Management	Feedback Management
User Registration	X						
User Login	X						
Product Catalog		X			X		

Product Search		X			X		
Product Details		X	X				
Shopping Cart			X		X		
Checkout Process	X						
Order Management			X	X	X		
Customer Services	X	X	X	X	X	X	X
Payment Processing			X	X	X		
Security And Privacy	X	X	X	X	X	X	X
Wish List			X				X
Recommendation Engine		X	X	X	X		
Reading Preview		X					
Reviews and Ratings	X	X				X	X
Inventory Management			X	X	X		
Discount And Promotions		X	X	X	X		
Shipping Options	X				X	X	
EBook Format	X	X					X
Gift Club	X	X					X
Social Media Integration	X						X